Peeking into the Future: MPC Resilient to Super-Rushing Adversaries*

Gilad Asharov† Anirudh Chandramouli* Ran Cohen‡ Yuval Ishai§

August 1, 2025

Abstract

An important requirement in synchronous protocols is that, even when a party receives all its messages for a given round ahead of time, it must wait until the round officially concludes before sending its messages for the next round. In practice, however, implementations often overlook this waiting requirement. This leads to a mismatch between the security analysis and real-world deployments, giving adversaries a new, unaccounted-for capability: the ability to "peek into the future." Specifically, an adversary can force certain honest parties to advance to round $r + 1$, observe their round $r + 1$ messages, and then use this information to determine its remaining round r messages. We refer to adversaries with this capability as "super-rushing" adversaries.

We initiate a study of secure computation in the presence of super-rushing adversaries. We focus on understanding the conditions under which existing synchronous protocols remain secure in the presence of super-rushing adversaries. We show that not all protocols remain secure in this model, highlighting a critical gap between theoretical security guarantees and practical implementations. Even worse, we show that security against super-rushing adversaries is not necessarily maintained under sequential composition.

Despite those limitations, we present a general positive result: secret-sharing based protocols in the perfect setting, such as BGW, or those that are based on multiplication triplets, remain secure against super-rushing adversaries. This general theorem effectively enhances the security of such protocols "for free." It shows that these protocols do not require parties to wait for the end of a round, enabling potential optimizations and faster executions without compromising security. Moreover, it shows that there is no need to spend efforts to achieve perfect synchronization when establishing the communication networks for such protocols.

*A preliminary version of this work appeared in EUROCRYPT 2025.

†Bar-Ilan University, {gilad.asharov,anirudh.chandramouli}@biu.ac.il. Research supported by the Israel Science Foundation (grant No. 2439/20), and by the European Union (ERC, FTRC, 101043243). Views and opinions expressed are however those of the author(s) only and do not necessarily reflect those of the European Union or the European Research Council. Neither the European Union nor the granting authority can be held responsible for them.

‡Reichman University, cohenran@runi.ac.il. Research supported in part by NSF grant no. 2055568 and by ISF grant 1834/23.

§Technion and AWS, yuvali@cs.technion.ac.il. Research supported by ISF grants 2774/20 and 3527/24, BSF grant 2022370, and ISF-NSFC grant 3127/23

Contents

1 Introduction

Secure multiparty computation (MPC) protocols [Yao86, GMW87, BGW88, CCD88] are typically studied in two main models: the *synchronous* model, where messages exchanged between honest parties are guaranteed to arrive within a known, bounded delay, and the *asynchronous* model, where messages can be delayed arbitrarily, and parties proceed independently at their own pace [BCG93, BKR94]. Though more general, the asynchronous model is often seen as overly pessimistic since such arbitrary delays are rare in practice, and networks, in most cases, are reliable. Consequently, much of the MPC literature focuses on the synchronous model, which allows for better resilience, simpler protocols, stronger security guarantees, and improved concrete efficiency.

In the synchronous model, parties begin the protocol at the same time and a fixed upper bound on message delivery time between parties is assumed to be known in advance, allowing the protocol to progress in predefined rounds. All messages in a given round must be delivered within this specified time limit, which must be respected throughout the protocol's execution. All parties are perfectly coordinated and proceed together from one round to the next.

In practice, however, achieving and maintaining truly synchronous communication is challenging. The synchronous model is idealized, and can be realized using global synchronization mechanisms, such as a common clock shared by all participants and bounded-delay channels [KMTZ13], but in the real world, parties may not be perfectly synchronized. Moreover, an implicit requirement in synchronous protocols is that even if a party receives all its messages for a given round early, it must wait until the round officially ends before sending the next-round's messages. This ensures that *all* honest parties receive their expected messages before proceeding. However, determining the end of a round, that is, establishing an appropriate message delivery bound, is problematic. Setting it too high results in inefficiency, while setting it too low risks security vulnerabilities or non-termination.

Unfortunately, in real-world implementations of MPC protocols, the requirement to wait (potentially idly) until the end of a round is often overlooked, and parties are instructed to proceed optimistically and send their round-$(r+1)$ messages as soon as they receive all their round-r messages. This creates a mismatch between security analyses and practical deployments. This mismatch could be an attempt to optimize efficiency, a result of ignorance, or a result of insufficient clarification in the way synchronous protocols are described in the literature, which separates the protocol layer and the communication layer.

This oversight introduces a potential security vulnerability. Specifically, it gives the adversary a new capability that is not accounted for in the security analysis. The adversary can "peek into the future" by sending round-r messages to an honest party P_j, causing P_j to proceed and send its round-$(r+1)$ messages (once P_j receives all other round-r messages). The adversary can then base its remaining round-r message choices for the corrupted parties on this "future" information. This is a strictly stronger capability than traditional "rushing," where the adversary only views round-r messages from honest parties before deciding the round-r messages for the corrupted parties.

We illustrate this adversarial capability with a toy example involving three parties: P_1, P_2, and P_3, running the following synchronous two-round protocol:

- **Round I:** P_1 picks and sends a random $x_1 \leftarrow \{0,1\}^\kappa$ to P_3, and likewise, P_2 picks and sends a random $x_2 \leftarrow \{0,1\}^\kappa$ to P_3 (where κ is the security parameter). All other messages are dummy (P_3 to both P_1, P_2, and messages between P_1 and P_2).
- **Round II:** P_1 sends x_1 to P_2 (and all other parties exchange dummy messages).

When the protocol is executed optimistically, P_2 can first send its round-I dummy message to P_1 and wait. Once P_1 receives the round-I message also from P_3, it proceeds to round II and sends

x_1 to P_2 (and other dummy messages to P_1). Now, P_2 can send $x_2 := x_1$ as its round-I message to P_3 and ensure that P_3 always receives the same message from both parties in round I. In contrast, such behavior is impossible in a standard synchronous protocol, where a malicious P_2 can ensure that $x_2 = x_1$ only with probability $2^{-\kappa}$, as x_1 and x_2 are chosen independently.

Super-rushing adversaries. We initiate a study of secure computation in the presence of *super-rushing adversaries* that have a limited ability to "peek into the future." Our goal is to bridge the gap between theoretical analyses of protocols and common practices in real-world implementations.

Our starting point is the synchronous model. The literature in this setting typically considers two types of adversaries with respect to their capability in ordering the message delivery: A *non-rushing* adversary must send the corrupted parties' messages in a given round *before* receiving their messages from the honest parties. A *rushing* adversary can arbitrarily order the message delivery within a round, enabling corrupted parties to choose their messages for the round *after* receiving the messages sent by honest parties.

In this work, we define stronger adversaries that we call *super-rushing*. Similarly to rushing adversaries, a super-rushing adversary can reorder messages within a given round, but, as opposed to rushing adversaries, it can also reorder messages *between different rounds*, i.e., deliver certain messages for round $r + 1$ before all messages for round r have been delivered. The only constraint is that an honest party must receive all of its round r messages before sending its round $r + 1$ messages.

We aim to understand the conditions under which existing synchronous protocols remain secure in the presence of super-rushing adversaries. That is, we ask whether the mismatch between the theoretical analysis and the implementation is of real concern. Our central motivating question is:

Do synchronous MPC protocols remain secure even in the presence of super-rushing adversaries?

Our primary goal is *not* to design new protocols tailored to this model, but to investigate whether existing synchronous protocols can run "as-is," without any modification. This contrasts with the work of Kushilevitz, Lindell, and Rabin [KLR06], who showed how to compile any perfect synchronous protocol into a secure protocol in the UC framework, which is inherently asynchronous (and thus allows such super-rushing attacks); this is achieved by changing the original protocol and adding a synchronization round after every communication round (thus doubling the round complexity). For further discussion, see Section 1.2.

Ideally, the hope is that since secure protocols are already robust against rushing adversaries and guarantee (at least) privacy and correctness, they will also remain secure against super-rushing adversaries. The best-case scenario is that the answer to the above question is affirmative. Such a result would have three important implications: First, for implementations that inadvertently bypass the requirement to wait until the end of the round, an affirmative answer would confirm that no real security risk is introduced. Second, for protocols that do enforce this waiting period, an affirmative answer would indicate that waiting is unnecessary, allowing for potential optimizations and faster execution without compromising security. Third, such a result would imply that there is no need to spend efforts to achieve perfect synchronization when establishing communication networks for secure protocols.

1.1 Our Results

We initiate a systematic study of protocols in the presence of super-rushing adversaries, with a focus on perfect security. At a very high level, our results show that not all protocols remain

secure against super-rushing adversaries, highlighting a critical gap between theoretical security guarantees and practical implementations. Nevertheless, our exploration finds sufficient conditions for which protocols remain secure in the presence of super-rushing adversaries. These conditions imply that central secure computation protocols from the literature maintain their security even in the presence of super-rushing adversaries. Thus, our results effectively enhance their security "for free." We believe that the same applies to almost all general-purpose synchronous MPC protocols from the literature, and our work can be seen as the first evidence in this direction.

Defining super-rushing adversaries. Before proceeding, we elaborate on the model. We consider synchronous protocols that are designed in a round-by-round manner and assume that communication between honest parties is private and reliable. As soon as P_j receives all the information it needs to compute the next-message function, P_j computes its outgoing messages and sends them. The adversary can control the message delivery according to its power: non-rushing, rushing, or super-rushing. Finally, recall that in a synchronous model, if P_i does not send a message to P_j, then P_j can recognize this;[1] hence, a standard modeling simplification is that if the adversary does not intend to send a message to P_j, to consider that the adversary instead sends some default message to P_j to indicate that. We carry this simplification over to our setting, as this naturally holds in our motivating scenario: when executing a synchronous protocol optimistically, the adversary cannot stall its messages without being detected.

We formalize super-rushing by letting the adversary "pull" messages by request (from honest parties to corrupted parties) and provide corrupted parties' messages in its own schedule within a round. The adversary is allowed to request round $r+1$ messages from an honest P_j under the restriction that it already sent all messages from corrupted parties to P_j in round r (see Section 3).

We also focus in this paper on protocols with all-to-all communication pattern in every round. This suffices to capture most of the multiparty protocols in the literature in the perfect setting, and already highlights the gap between super-rushing and rushing. On the other hand, in such protocols, the adversary can potentially see just one round into the future (due to all-to-all communication, the gap between every pair of honest parties can be at most one round). Our results naturally extend to message-oblivious protocols [CMS85, CK93], where the communication pattern is predetermined and does not depend on inputs or randomness, even though in such protocols the adversary may have the ability to see beyond a single round into the future.

Single-input functionalities. As a warmup, we first investigate protocols where a single party holds an input. Despite being a simplified variant, such functionalities are of high importance. Single-input functionalities include broadcast protocols [PSL80, LSP82], where a sender wishes to distribute a message to all participants (and the parties must agree on the message received). Another important functionality is verifiable secret sharing (VSS) [CGMA85], where a dealer distributes a secret to n participants while the parties can verify that a well-defined secret was shared. Moreover, it allows, for instance, the distribution of "multiplication triplets with a dealer," a highly important building block in perfect MPC (e.g., [BGW88]). It allows a dealer to distribute polynomials $A(x), B(x), C(x)$, such that each party P_i receives $A(i), B(i), C(i)$, while all parties guarantee that $A(x), B(x), C(x)$ are of degree-t, and that $C(0) = A(0) \cdot B(0)$. Using this primitive, parties generate multiplication triplets [Bea91]. Our first result shows that:

Theorem 1.1 (Informal)**.** *Let f be a single-input functionality, and consider a protocol π that perfectly securely realizes f in the presence of a rushing adversary. Moreover, assume that:*

[1] This can be achieved using timeouts when instantiating synchronous communication via a common clock and bounded-delay channels.

- *If the input provider is corrupted, then the input x can be extracted from the joint view of the honest parties;*
- *If the input provider is honest, then for every x and x' such that $f_I(x) = f_I(x')$ (where f_I denotes the outputs of the corrupted parties), the view of the corrupted parties in an execution of π with input x is identical to their view in an execution with x'.*

Then, π perfectly realizes f in the presence of a super-rushing adversary.

Note that verifiable secret sharing, generation of multiplication triplets, or functionalities with no privacy (such as broadcast), satisfy the conditions of Theorem 1.1, as well as other results in the literature, e.g., [GIKR02, ALR11].

In fact, we prove an even stronger result than stated in Theorem 1.1: we can relax the requirements from the protocol and assume security against just a *non-rushing* adversary (which, in round r first provides its messages and only then receives the honest parties' round r messages). This suffices to automatically upgrade the security twice—ensuring security against both rushing and super-rushing adversaries, at no additional cost! In other words, when proving the security of such protocols in the past, we were overextending our efforts yet underestimating the results.

Two-input functionalities. After establishing that super-rushing is of no concern for single-input functionalities, we turn our attention to functionalities with two input providers. Specifically, we consider the simultaneous broadcast functionality [CGMA85], denoted as f_{sb}, where two designated parties, say P_1 and P_2 holding bits b_1 and b_2, respectively, wish to broadcast those bits in parallel, i.e., the output of the all participants should be (b_1, b_2). The critical security requirements here is *independence of inputs*, ensuring that no sender can choose its bit as a function of the other input. Here, things dramatically change:

Theorem 1.2 (informal)**.** *There exists a protocol that implements the two-input functionality f_{sb} which is secure against a rushing adversary but is insecure against a super-rushing adversary.*

We consider the five-party protocol from [GIKR02], where in round 1 parties P_1 and P_2 send their bit to parties P_3, P_4, and P_5. In round 2, each of those parties echo the pair of bits it received to everyone, who then take the majority as their output. Intuitively, the super-rushing adversary, corrupting, say, P_1, can "peek into the future" and learn the bit b_2 of the other sender: this can be done by sending 0 to P_3 who will echo the pair $(0, b_2)$ to everyone including P_1 (here we rely on security against a single corruption, meaning that if P_3 is cheating with this behavior, it cannot affect correctness). Next, P_1 can pick its own input b_1 as a function of b_2 (e.g., set $b_1 = b_2$). That is, P_1 breaks the *independence of inputs* requirement of secure computation.

A closer look at this protocol reveals that the protocol does not have a "committal round" [MR92, DM00], which is a predefined round during which the parties' inputs become fixed (preventing the adversary from altering its input beyond this point) while still hiding all information about the output. This leads to the conjecture of whether the existence of a committal round is a sufficient condition for guaranteeing resiliency to super-rushing. However, it does not:

Theorem 1.3 (informal)**.** *There exists a protocol for f_{sb}, admitting a committal round* CR*, that is secure against a rushing adversary but is insecure against a super-rushing adversary.*

In this negative result, the protocol is very natural and consists of a verifiable secret sharing phase (in which P_1 and P_2 "commit" to their respective input bits b_1 and b_2), followed by an "output revealing round," in which the parties learn the output. This "commit-then-reveal" structure is the standard approach for implementing coin-tossing mechanisms. As such, this result demonstrates that central building-blocks for MPC might go wrong when considering super-rushing adversaries.

Sequential composition. We proceed to study sequential composition of protocols that are secure against super-rushing. Somewhat surprisingly, we show that even a sequential composition of two single-input protocols does not necessarily remain secure against a super-rushing adversary.

Theorem 1.4. *There exists a 4-ary single-input functionality f and a 4-party protocol π_f that securely realizes f against a super-rushing adversary corrupting a single party. Further, there exists a 4-ary functionality g and a 4-party protocol π in the f-hybrid model that consists of 2 calls to f and securely realizes g against a super-rushing adversary corrupting a single party. Nevertheless, the protocol $\pi^{f\to\pi_f}$ that replaces the calls to f with the execution of π_f does not securely realize g in the presence of a super-rushing adversary corrupting single party.*

General possibility result. In our negative results, we see that the adversary has the capability to peek into the future, learn something, and use this information to change its own input accordingly. Which protocols prevent such a behavior?

Our primary contribution is a general positive result, demonstrating that any perfectly secure protocol against a rushing adversary that satisfies certain natural conditions remains secure even against a super-rushing adversary. In other words, although the adversary can "peek into the future," this additional power does not compromise security.

These "natural conditions" required by our transformation align closely with the structure found in most perfect, secret-sharing-based secure protocols, such as the celebrated protocol of Ben Or, Goldwasser, and Wigderson [BGW88]. The BGW protocol operates in three phases: (1) Input sharing phase (VSS)—by the end of this phase, the inputs of all parties are fixed and no information about the output is revealed; (2) Circuit emulation phase—the parties emulate the computation of the circuit gate-by-gate, computing shares on the output wire of each gate based on the shares of the input wires; notably, during this phase, the simulator can carry out the simulation without needing to know the final outputs of the computation; and (3) Output reconstruction phase – the outputs are reconstructed (and are needed for the simulation of this phase).

Informally, our approach requires the protocol to include a "committal round" (denoted CR), a round in which the inputs are fixed and committed but the output remains hidden, and an "output revealing round" (denoted ORR), a round during which the outputs are disclosed. In BGW, the committal round is the end of the "input sharing phase," and the output revealing round is the last round, where the parties learn the outputs.

Recall that the negative example in Theorem 1.3 has a CR (round 1) and an ORR (round 2), which may sound contradicting to the discussion above; however, in this example it holds that $\mathsf{ORR} = \mathsf{CR}+1$. Our first positive result complements this example by showing that for all protocols satisfying $\mathsf{ORR} > \mathsf{CR} + 1$, that is, protocols for which "the circuit emulation" phase takes at least one round, super-rushing security follows from rushing security.

Theorem 1.5 (informal)**.** *Every perfectly secret-sharing-based secure protocol that (1) has all-to-all communication, (2) admits a committal round* CR*, (3) admits an output revealing round* ORR*, and (4) satisfies* $\mathsf{ORR} > \mathsf{CR} + 1$*, is secure against a super-rushing adversary.*

We emphasize that this statement is informal, and there are several other requirements from the protocol that are not specified in the above statement. See Section 6 for the formal treatment.

Intuitively, the adversary can "peek into the future" by observing the next communication round. However, due to the all-to-all communication pattern, this peeking is restricted to just one round ahead. The adversary, thus, might look into round $\mathsf{CR} + 1$ while some parties have not yet completed their committal round. The condition $\mathsf{ORR} > \mathsf{CR} + 1$ ensures that the simulator can continue constructing the adversary's view at this point without relying on the actual inputs or

outputs. By the time the adversary peeks into round ORR, where the outputs are actually required, all parties are guaranteed to have completed their committal round. At this point, the adversary can no longer cause any harm.

Theorem 1.5 is very powerful. It already captures natural protocols in the literature, such as BGW for non-linear circuits [BGW88], protocols that follow a similar paradigm [CCD88, BB89, FY92, HM97, GRR98, HM00, CDM00, ALR11, CCXY18, AAY21], and protocols that are based on the online-offline model (e.g., [Bea91, HMP00, HM01, BH08, CP17, AAPP23]). Our proof upgrades the security of all those protocols, for free, and without any modifications.

Extensions. We extend Theorem 1.5 in two central ways:

- We show that it is easy to transform any protocol that has a CR and an ORR but for which $\mathsf{ORR} = \mathsf{CR} + 1$ to be secure against super-rushing adversaries, by adding one more "dummy" round between CR and ORR.
- We consider the special case where the CR is over the broadcast channel (in which case if the adversary progresses party P_j to round $r+1$, it "commits" to the message is would send to all other honest parties in round r). In that case, $\mathsf{ORR} = \mathsf{CR}+1$ suffices to guarantee security against super-rushing adversaries. This captures, for instance, the BGW protocol for linear functions, in which the final round of its VSS is a vote over the broadcast channel, and there is no need to add a synchronization round to BGW in that case. We conclude that BGW remains secure against super-rushing, regardless of whether the function is linear or not. This extension also captures the perfectly secure protocols of [ABT19, AKP20] for NC^1 circuits and upgrades their security against super-rushing adversaries.

Statistical setting. After establishing an understanding of super-rushing in the perfect setting, we turn our attention to the statistical setting. We show that unfortunately, a protocol that satisfies all the requirements of Theorem 1.5, i.e., is secret-sharing based, admits a committal round, admits an output revealing round, and satisfies $\mathsf{ORR} > \mathsf{CR} + 1$, but is only *statistically secure*, does not provide security in the presence of a super-rushing adversary, although it is secure against a rushing adversary.

In fact, our negative results complement each one of the conditions of Theorem 1.5: We showed a negative result for a protocol that does not have a CR (Theorem 1.2); When there is a CR and an ORR but for which $\mathsf{ORR} = \mathsf{CR} + 1$ (Theorem 1.3); or when the protocol admits a CR, an ORR, and satisfies $\mathsf{CR} > \mathsf{ORR} + 1$ but is only statistically secure. Nevertheless, we emphasize again that the statement of Theorem 1.5 is simplified, and there are additional necessary requirements from the simulator of the rushing adversary that are not reflected in the informal statement.

Open questions. We believe that a more systematic study of super-rushing is an interesting and well-motivated topic for future work. Our work leaves open several natural questions. What about protocols that do not have an all-to-all communication pattern, such as protocols with a "king-phase" communication pattern (such as [DN07, GLS19])? Under what conditions composition of protocols secure against super-rushing remain secure? Which protocols remain secure in the statistical or computational setting?

Despite the seemingly pessimistic tone of our negative result, we conjecture that almost all, if not all, general-purpose protocols outside the scope of our current framework still remain secure against super-rushing adversaries. In particular, the negative result we present for the statistical setting is somewhat contrived, as it introduces a negligible error that is almost never exploitable by traditional rushing adversaries but becomes amplified and noticeable under super-rushing attacks. The nature

of this error in statistically secure protocols, and even in computationally secure protocols, is fundamentally different and does not seem to be affected by super-rushing adversaries, but we leave it for further exploration.

1.2 Related Work

Fully asynchronous model. The Universal Composability (UC) framework, introduced by Canetti [Can01], operates under a fully asynchronous and adversarial communication model, without any guarantees on message delivery. In this model, the adversary has complete control over the network, meaning that when one honest party sends a message to another, it may never reach its destination. This stands in contrast to our model, where reliable message delivery is assumed. Consequently, the UC framework inherently does not provide guaranteed termination, as the adversary has the ability to entirely block communication. Indeed, the adversary has inherent capabilities to "peek into the future." For example, it might observe future messages from one honest party P_j (if P_j has advanced in the protocol) while another party P_k lags behind.

Kushilevitz, Lindell, and Rabin [KLR06] introduced a transformation that converts any information-theoretically secure protocol in the stand-alone, synchronous model (satisfying a few basic properties) into one that is secure in the UC setting. Their transformation alters the original protocol by introducing a synchronization round after every protocol round, where each party announces to all when it is ready to move on to the next round, and proceeds once reeving announcements from everyone. This indeed protects against super-rushing attacks, but effectively doubles the number of rounds required, and blows up the message complexity of each round to quadratic (even if the underlying protocol does not require so). Further, this approach inherently cannot guarantee termination, as a single corrupted party can block the entire computation by remaining silent. Our focus, in contrast, is on evaluating whether existing protocols remain secure as is, *without* any modification.

Several works establish synchronous communication over asynchronous networks [Can01, Nie03, HM04, KLP07, KMTZ13, LM20, BDD+21]. Protocols in these models are designed in a lock-step manner, which prevents super-rushing attacks. However, such protocols may be vulnerable to super-rushing adversaries when executed optimistically.

Asynchronous setting with eventual delivery. A long and fruitful line of work in the realm of distributed computing studies asynchronous protocol with eventual message delivery; that is, where the adversary can arbitrarily delay messages, though all messages must eventually be delivered. As such, adversaries in this model also have limited ability to "peek into the future." Ben-Or, Canetti, and Goldreich [BCG93] initiated the study of MPC over asynchronous networks with eventual delivery (see, e.g., [BCG93, BKR94, HNP05, Coh16, CGHZ16, AAPP24] and references therein). The inherent challenge in this model is that one cannot distinguish between an honest party whose message is delayed from a corrupted party who does not send the message at all. This uncertainty leads to certain impossibility results and reduced resilience compared to the synchronous model: e.g., impossibility of deterministic protocols for Byzantine agreement [FLP85], impossibility of asynchronous Byzantine agreement and MPC with $t \geq n/3$ (even assuming cryptography) [DLS88, BCG93], and impossibility of perfect asynchronous MPC with $t \geq n/4$ [BCG93]. In contrast, our model assumes that the corrupted parties always send messages when suppose to, which is a realistic assumption when optimistically executing a synchronous protocol. Therefore, these impossibility results do not apply in our setting.

Hybrid settings. Several works have looked at models that combine synchrony and asynchrony in various settings. A line of work initiated by Blum, Katz, and Loss [BKL19] studies protocols that can obtain a certain security assuming synchrony, yet satisfy weaker security guarantees if this assumption does not hold and the communication is asynchronous; see, e.g., [BZL20, BCLL23, ACC25]. This model enables running synchronous protocols while ensuring security against super-rushing adversaries. However, as opposed to our results, such protocols achieve *weaker* security if the adversary is super-rushing (and, in particular, are limited by the lower bounds of asynchronous MPC); further, the goal in these works is different from our motivation of studying MPC protocol designed for the synchronous setting *without modifications*.

Another line of work, initiated by Beerliová-Trubíniová, Hirt, and Nielsen [BHN10], studies protocols in which the protocol begins with synchronous rounds and proceeds with asynchronous communication. Such protocols may be vulnerable to super-rushing adversaries if the synchronous part uses point-to-point channels (as opposed to a broadcast channel) and is executed optimistically.

Optimistic responsiveness. A goal, similar to ours, is to run protocols in a synchronous network, but proceeds as fast as the actual network delays allow (as opposed to the worst-case delays). In the context of consensus, Attiya et al. [ADLS91] studied Byzantine agreement secure against crashes in this model, and Pass and Shi [PS17, PS18] studied state machine replication, and showed a lower bound of $t < n/3$ corruptions applies also to this setting (see also [SARN20]). Liu-Zhang et al. [LLM+20] constructed MPC protocols that achieve optimistic responsiveness, by compiling together an asynchronous MPC and a synchronous MPC protocols. Our work enables achieving optimistic responsiveness for synchronous protocols *without modifications*, as long as all parties send their messages.

2 Technical Overview

In this section, we highlight some of our results while trying to give some intuition as to what might go wrong and when one can expect that super-rushing does not introduce security vulnerability.

What might go wrong? We start with one of our negative results, demonstrating the advantage a super-rushing adversary gains. Specifically, we consider the protocol and the adversary underlying Theorem 1.3, that admits a committal round CR, but for which $\mathsf{ORR} = \mathsf{CR} + 1$.

Consider the following simultaneous broadcast functionality between 5 parties where 2 parties provide inputs (and λ denotes the empty string):

$$f_{\mathsf{sb}}(b_1, b_2, \lambda, \lambda, \lambda) = (y, y, y, y, y) \qquad \text{where} \qquad y = (b_1, b_2).$$

To implement this functionality, we consider a protocol, denoted $\pi_{\mathsf{sb\text{-}vss}}$, in which P_1 and P_2 first use verifiable secret sharing (VSS) to distribute shares of b_1 and b_2, respectively, and then the parties jointly reconstruct those secrets to learn b_1 and b_2. Intuitively, this guarantees independence of inputs assuming synchronous communication and rushing adversaries: each one of the input providers must "commit" to its input before learning the other input. We remark that this is also the standard way to toss a coin: P_1 and P_2 can choose their secrets uniformly at random, commit to them, reveal them, and take a combination of the results.

Gennaro, Ishai, Kushilevitz, and Rabin [GIKR01] showed a 1-round VSS for 5 parties secure against one corruption. Specifically, the dealer distributes a secret s using a polynomial with degree 1. In the reconstruction phase, the dealer is not allowed to participate, and each party that receives a share simply forwards it to all others. The parties run Reed-Solomon decoding on the

shares received. If there is no unique decoding (meaning that the dealer is cheating), the parties output a default value. Observe that:

- If the dealer is honest, then it must have sent valid shares to all parties, and privacy holds since $t = 1$ (and we assume just one corrupted party). During the reconstruction, the adversary can introduce only one error, which is eliminated using Reed Solomon decoding.
- If the dealer is corrupted and sent only one incorrect share (i.e., provided 3 shares that lie on the same line, and 1 that does not), then reconstruction will be successful. Specifically, since the participants are honest and forward the messages they received from the dealer, then all parties will have just a single error and 3 correct points, and therefore the single error can be eliminated using the Reed Solomon decoding. If the dealer distributed more than a single error, then no party will be able to decode, and all parties will output a default value.

This protocol $\pi_{\mathsf{sb\text{-}vss}}$ indeed perfectly implements the f_{sb} in the presence of a rushing adversary. However, it does not remain secure in the presence of a super-rushing adversary, who corrupts, say, P_1:

- In the first round, the honest P_2 shares its input b_2 to four shares s_1, s_3, s_4, s_5 by giving each party P_i its share s_i, for $i \in \{1, 3, 4, 5\}$.
- After the corrupted P_1 receives a share s_1 from P_2, party P_1 sends to some honest party, say P_3, a random point in the field as its "share."
- At this point, P_3 receives all its round-1 messages: a share from P_1 and a share from P_2, and proceeds to the next round—the reconstruction phase. In this phase, P_3 forwards the two shares it received to everyone (and, in particular, sends s_3 to everyone).
- The adversary P_1 receives s_3, and together with s_1 that it received directly from the dealer, P_2, it can reconstruct b_2.
- Finally, P_1 chooses b_1 as a function of b_2, and provides valid shares for b_1 to all parties. Note that it can even choose the polynomial such that it would even agree with the share it already provided to P_3, seemingly providing no error whatsoever!

The above attack shows that things might go wrong, and the super-rushing adversary can break the "independence of inputs" requirement in secure protocols. It is important to note also that the above protocol does have a "committal round" when considering rushing adversaries—there is a specific round in the protocol (round 1) in which the inputs of all parties (that provide inputs) is well-defined. As we show, the problem is that the "output revealing round" is too close to the committal round, and the ability of super-rushing adversaries that can progress some parties to the next round, enables learning the output *before* the conclusion of the committal round.

Single-input functionalities. A specific class of functionalities where "independence of inputs" is not a concern is the case where only one party provides an input. As a warmup towards our general positive result, we consider such functionalities first. We show that for single-input functionalities, any protocol that is secure against non-rushing adversaries, is already secure against super-rushing adversaries.

For simplicity, let us assume in this technical overview that the input provider is honest and holds input x. In the ideal model, the input provider sends its input to the trusted party, the trusted party computes $f(x) = (y_1, \ldots, y_n)$, and gives each honest party P_j its output y_j. The ideal adversary (the simulator) cannot influence this execution in any way. Focus for now just on the correctness of the protocol. The real-world adversary, has this extra capability of "peeking

into the future." The question is whether this ability can be used to break the correctness of the protocol, that is, to cause some P_j output a value that is not y_j.

The key reason why super-rushing is not a concern for this class of functionalities, is that we can simulate round $r+1$ message of every honest party. Specifically, we construct from the super-rushing adversary exponentially many adversaries—one for every possible execution that might occur with this particular super-rushing $\mathcal{A}'$. For every possible combination of random tapes of the honest parties $(\rho_j)_{j\notin I}$ (where I denotes the set of corrupted parties), we construct a rushing adversary $\mathcal{A}[(\rho_j)_{j\notin I}]$. This rushing adversary internally simulates the protocol, with each honest party P_j using ρ_j as its random tape (and no input), while the input provider input x, and interacting with the super-rushing adversary $\mathcal{A}'$. The rushing adversary then constructs all expected incoming and outgoing messages of the protocol in each round for all parties. After generating these messages, $\mathcal{A}[(\rho_j)_{j\notin I}]$ executes the protocol with the real honest parties. If, by chance, the actual honest parties use the same random tapes $(\rho_j)_{j\notin I}$, the adversary knows exactly how to behave and what messages to send in each round. Otherwise, the adversary aborts.

The key insight is that if the super-rushing adversary $\mathcal{A}'$ were to gain any advantage (i.e., break correctness), there would exist some execution where one of these rushing adversaries $\mathcal{A}$ also gains an advantage. However, the latter is impossible because of the *perfect* security of the protocol. The above intuition addresses just correctness, but a similar argument holds for other security properties. The formal proof also addresses a full simulation of the super-rushing adversary and the scenario where the input provider is corrupted. For the full details, we refer the reader to Section 5.

Crucially, the above argument breaks down when there are two input providers. In that case, the ideal world requires this extra property of "independence of inputs." The real-world adversary can peek into the future, learn some information about the input of the honest party, and use this information to correlate its input. Such behavior is possible in the real world but is not allowed in the ideal world.

General functionalities: our main possibility result. We now turn our attention to the crux of our contribution, which is a possibility result for general functionalities. We focus in this overview on the simpler case—protocols that admit a committal round CR and an output revealing round ORR, but for which $\mathsf{ORR} > \mathsf{CR} + 1$.

When considering rushing adversaries, the existence of CR and ORR guarantee the following in the real execution:

- **Committal round:** CR is the point in the execution where the inputs of all parties are committed, and the adversary can no longer influence the outputs of the honest parties after that point.
- **Output revealing round:** There is no leakage on the honest parties inputs' until ORR. That is, ORR is the first (and only point) in the execution where the parties learn the outputs, and the adversary learns at that point the information about the honest parties inputs' that is implied from those outputs.

Intuitively, in the real world, the adversary does have some extra power to look into future rounds. However, such protocols provide the adversary $\mathcal{A}'$ with no information on the outputs until ORR. Moreover, the adversary cannot look "too much" into the future, without progressing the protocol and providing messages. When $\mathsf{CR} > \mathsf{ORR} + 1$, the guarantee is that when $\mathcal{A}'$ peeks into round ORR, then $\mathcal{A}'$ must have already provided its inputs in CR, and therefore, $\mathcal{A}'$ cannot influence anything at this point.

To actually simulate the super-rushing adversary $\mathcal{A}'$, we use the simulator of the rushing adversary. The technical difficulty in the proof is that we need to invoke this underlying simulator multiple times on different behaviors of (rushing) adversaries and to show that all the invocations are consistent.

In more detail, the super-rushing adversary $\mathcal{A}'$ might ask for round $r+1$ messages of some honest party P_j before it actually provided all the corrupted parties' messages for round r. The simulator $\mathcal{S}'$ of $\mathcal{A}'$ then feeds the rushing simulator $\mathcal{S}$ of all round-r messages that were sent from corrupted parties to P_j, and with dummy messages from the corrupted parties to the other honest parties. The simulator $\mathcal{S}'$ obtains back all the round $r+1$ messages that the honest parties send to the corrupted parties, and extracts from this the messages that P_j sends in round $r+1$. Based on this information, the adversary might provide messages from corrupted parties to honest parties in round r. We then will have to restart the simulator $\mathcal{S}$ and provide it with updated information of corrupted parties to honest parties in round r. However, since all the messages that P_j has received in round r are identical between the two executions of $\mathcal{S}$, it must provide the exact same outgoing messages in round $r+1$ for P_j.

Our proof assumes some structure of the underlying simulator $\mathcal{S}$ that is used to simulate rushing adversaries. Nevertheless, we show that such requirements are quite natural and are satisfied for several protocols in the literature. We refer the reader to Section 6 for the formal treatment.

Organization. The rest of the paper is organized as follows. After the preliminaries in Section 3, we provide some separations between rushing and super-rushing in Section 4. Single-input functionalities are discussed in Section 5. The main theorem is given in Section 6, and we discuss the statistical setting in Section 7.

3 Preliminaries

Protocols. An n-party protocol $\pi = (P_1, \ldots, P_n)$ is an n-tuple of PPT interactive Turing machines (ITM). The term *party* P_i refers to the i^{th} ITM. Each party P_i starts with input $x_i \in \{0,1\}^*$ and random coins $\rho_i \in \{0,1\}^*$. An *adversary* $\mathcal{A}$ is another ITM describing the behavior of the corrupted parties. We consider malicious, computationally unbounded adversaries, that can deviate from the protocol in an arbitrary way. For simplicity we focus on static corruptions, meaning that the adversary decides on the set of parties to corrupt before the beginning of the execution.[2] That is, the adversary starts the execution with input that contains the identities of the corrupted parties, their private inputs, and an additional auxiliary input z.

3.1 Communication Model

As standard for information-theoretic MPC, we assume that each party can communicate with every other party over an ideally secure communication channel, meaning that the adversary cannot eavesdrop nor alter honest-to-honest communication.

Synchronous communication. We say that a protocol π is executed over a synchronous communication network if the protocol proceeds in a lock-step manner, round by round. That is, there is a global round counter that is known to all parties and advances in a sequential manner. Each round consists of a *send phase* (where parties send their messages for this round) followed by a

[2] This makes our separations stronger, and in the perfect-security setting any statically secure protocol satisfying a few basic requirements is also adaptively secure [CDD+01, DN14, ACS22].

receive phase (where they receive messages from other parties). All parties begin the protocol at round 1, and for every round $r \in \mathbb{N}$, all parties advance from round r to round $r+1$ in a synchronized way in the sense that no party advances to round $r+1$ before all honest parties have completed their round r operations. Specifically, it is guaranteed that every message sent in round r is delivered to its destination, and this is done before any message is sent for round $r+1$. When a party P_i sends a message m to party P_j in round r, we denote the message together with the metadata by (i, j, r, m).

We will also consider protocols that are defined in the broadcast model. Here, in every round each party may send private messages using the point-to-point channels, and also broadcast a message that will arrive to all parties in the same round.

In every round, every party can send a message to any other party as a function of its input, its randomness, its history of incoming messages, and the identity of the recipient. For simplicity, and unless stated otherwise, we will consider protocols where in each round there is all-to-all communication, i.e., every party sends a (possibly empty) message to every other party (either over point-to-point channels or via the broadcast channel), and where the number of rounds is fixed and known ahead of time. Our results can be extended to *message-oblivious* protocols [CMS85, CK93] which means that the decision of whether party P_i should send a message to party P_j in round r is determined by the protocol, regardless of the input or the random tape. We further assume that every corrupted party always sends a message when supposed to (this is a standard assumption for synchronous protocols, as the recipient can identify the event where a party is cheating by not sending a message when it is supposed to, and can pretend receiving a default message instead).

We note that in a synchronous, message-oblivious protocol, each honest party knows how many messages it should receive in a given round and from whom. As such, every honest party can generate its messages for round $r+1$ immediately after receiving all of its messages for round r. The synchrony assumption, however, ensures that those messages are delivered to their destinations only after all round-r messages have been delivered to all honest parties.

Rushing and non-rushing adversaries. In the synchronous setting, two common types of adversaries are considered in the literature: *rushing* and *non-rushing*. A rushing adversary has the ability to schedule the order in which messages are delivered within a given round; as such, the corrupted parties can choose their messages to be sent in round r after receiving the messages sent by honest parties to corrupted parties in round r. That is, in round r, the adversary initially obtains all messages of the form $(j, i, r, \cdot)$ for an honest P_j and a corrupted P_i, and based on this information produces the messages $(i, j, r, \cdot)$ for each corrupted P_i to each an honest P_j.

In contrast, a non-rushing adversary must choose the corrupted parties' messages for round r independently of the messages sent by honest parties in round r. That is, in round r, the adversary initially generates all the messages $(i, j, r, \cdot)$ from a corrupted P_i and an honest P_j and only later obtains the messages of the form $(j, i, r, \cdot)$ for every honest P_j to every corrupted P_i.

We emphasize the both for rushing and non-rushing adversaries, it is guaranteed that every message of the form $(i, j, r, \cdot)$ is delivered at its destination before the delivery of any message of the form $(i', j', r+1, \cdot)$.

Super-rushing adversaries. We will also consider stronger adversaries that we call *super-rushing*. Similarly to rushing adversaries, a super-rushing adversary can reorder messages within a given round, but, as opposed to rushing adversaries, it can also reorder messages *between different rounds*, i.e., deliver certain messages for round $r+1$ before all messages for round r have been delivered.

Effectively, in an execution of a message-oblivious protocol, an honest party can generate its messages for round $r+1$ immediately after receiving all of its messages for round r; a super-rushing adversary can then deliver some of those messages before all honest parties have received all their messages for round r.

The interface of the adversary. For concreteness, and to simplify notations in our proofs, we will use the following interface between the adversary and the honest parties:

- The adversary can send a message from a corrupted party to an honest party.
- The adversary can request a message from an honest party to a corrupted party.

We assume that the adversary is well behaved in the following sense. First, the adversary always sends a message from a corrupted party to an honest party when supposed to, and second, if the adversary requests a message corresponding to $(j^*, i^*, r+1)$ for an honest P_{j^*} and a corrupted P_{i^*}, then the following holds:

- If the adversary is non-rushing, then the adversary must have sent all of the messages corresponding to (i, j, r') for every corrupted P_i, honest P_j, and $r' \leq r+1$.
- If the adversary is rushing, then the adversary must have sent all of the messages corresponding to (i, j, r') for every corrupted P_i, honest P_j, and $r' \leq r$.
- If the adversary is super-rushing, then the adversary must have sent all of the messages corresponding to (i, j, r') for every corrupted P_i, honest P_j, and $r' \leq r-1$, and also all of the messages corresponding to (i, j^*, r') for every corrupted P_i and $r' = r$.

We remark that the above requirement for super-rushing is for the case of all-to-all communication. In general, the requirement is that the adversary can request a message $(j^*, i^*, r+1)$ if P_{j^*} has received all the messages expected to arrive by the protocol in round r (recall that in the general case, we assume the protocol is message-oblivious, and so the communication pattern is fixed).

3.2 Secure Protocols

We consider the standard definition of secure multiparty computation according to the real/ideal paradigm (see [Can00, Gol04] for further details).

Real-world execution. We consider an n-party protocol $\pi = (P_1, \ldots, P_n)$, where each party P_i starts with input $x_i \in \{0,1\}^*$ and random coins $\rho_i \in \{0,1\}^*$. The adversary $\mathcal{A}$ starts the execution with input that contains the identities of the corrupted parties $I \subseteq [n]$, their private inputs, and an additional auxiliary input z.

We consider synchronous, message-oblivious protocols as defined above, where every pair of parties is connected by an ideally secure communication channel. Whenever an honest party receives all the messages it is supposed to receive for some round r, the party generates its messages for round r. The scheduling of the message delivery depends on the type of the adversary: non-rushing, rushing, or super-rushing (as defined in Section 3.1).

Throughout the execution of the protocol, all the honest parties follow the instructions of the prescribed protocol, whereas the corrupted parties receive their instructions from the adversary, that can instruct the corrupted parties to deviate from the protocol in any arbitrary way (however, we assume that corrupted parties will send some message to an honest party for a given round if they are supposed to). At the conclusion of the execution, the honest parties output their prescribed output from the protocol, the corrupted parties do not output anything and the adversary outputs

its view of the computation, containing the views of all corrupted parties. The view of a party in a given execution of the protocol consists of its input, its random coins, and the messages it receives throughout this execution.

Definition 3.1 (Real-world execution)**.** *Let $\pi = (P_1, \ldots, P_n)$ be an n-party protocol and let $I \subseteq [n]$ denote the set of indices of the parties corrupted by $\mathcal{A}$. Denote by $\text{REAL}_{\pi,I,\mathcal{A}}(\boldsymbol{x}, z, \boldsymbol{\rho})$ the output vector of $P_1, \ldots, P_n$ and $\mathcal{A}$ resulting from the protocol interaction on input vector $\boldsymbol{x} = (x_1, \ldots, x_n)$, auxiliary input z, and randomness $\boldsymbol{\rho} = (\rho_{\mathcal{A}}, \rho_1, \ldots, \rho_n)$. Let $\text{REAL}_{\pi,I,\mathcal{A}}(\boldsymbol{x}, z)$ denote the probability distribution of $\text{REAL}_{\pi,I,\mathcal{A}}(\boldsymbol{x}, z, \boldsymbol{\rho})$ where $\boldsymbol{\rho}$ is uniformly chosen. Let $\text{REAL}_{\pi,I,\mathcal{A}}$ denote the distribution ensemble $\{\text{REAL}_{\pi,I,\mathcal{A}}(\boldsymbol{x}, z)\}_{(\boldsymbol{x},z)\in(\{0,1\}^*)^{n+1}}$.*

Ideal-world execution. The ideal-world computation is parameterized by a (potentially randomized) n-ary functionality $f : (\{0,1\}^*)^n \to (\{0,1\}^*)^n$ to compute. An ideal computation of f on input $\boldsymbol{x} = (x_1, \ldots, x_n)$ for parties $(P_1, \ldots, P_n)$ in the presence of an adversary (a simulator) $\mathcal{S}$ controlling the parties indexed by $I \subseteq [n]$, proceeds via the following steps:

***Sending inputs to trusted party*:** An honest party P_i sends its input x_i to the trusted party. The adversary may send to the trusted party arbitrary inputs for the corrupted parties. Let x_i' be the value actually sent as the input of party P_i.

***Computation stage*:** If x_i' is outside of the domain for P_i or if P_i sends no input, the trusted party sets x_i' to be some predetermined default value. Next, the trusted party samples randomness $\rho_f \leftarrow \{0,1\}^*$, computes $(y_1, \ldots, y_n) = f(x_1', \ldots, x_n'; \rho_f)$, and sends y_i to party P_i.

***Outputs*:** Honest parties always output the message received from the trusted party while the corrupted parties output nothing. The adversary $\mathcal{S}$ outputs an arbitrary function of the initial inputs $\{x_i\}_{i\in I}$, the messages received by the corrupted parties from the trusted party, and its auxiliary input.

Definition 3.2 (ideal computation: static case)**.** *Let $f : (\{0,1\}^*)^n \to (\{0,1\}^*)^n$ be an n-ary functionality and let $I \subseteq [n]$ be the set of indices of the corrupted parties. Denote by $\text{IDEAL}_{f,I,\mathcal{S}}(\boldsymbol{x}, z, \boldsymbol{\rho})$ the output vector of $P_1, \ldots, P_n$ and $\mathcal{S}$ resulting from the above described ideal process on input vector $\boldsymbol{x} = (x_1, \ldots, x_n)$, auxiliary input z, and randomness $\boldsymbol{\rho} = (\rho_f, \rho_{\mathcal{S}})$. Let $\text{IDEAL}_{f,I,\mathcal{S}}(\boldsymbol{x}, z)$ denote the probability distribution of $\text{IDEAL}_{f,I,\mathcal{S}}(\boldsymbol{x}, z, \boldsymbol{\rho})$ where $\boldsymbol{\rho}$ is uniformly chosen. Let $\text{IDEAL}_{f,I,\mathcal{S}}$ denote the distribution ensemble $\{\text{IDEAL}_{f,I,\mathcal{S}}(\boldsymbol{x}, z)\}_{(\boldsymbol{x},z)\in(\{0,1\}^*)^{n+1}}$.*

Security definition. Having defined the real and ideal computations, we can now define static security of protocols.

Definition 3.3 (perfect security)**.** *Let $f : (\{0,1\}^*)^n \to (\{0,1\}^*)^n$ be an n-ary functionality. A protocol π statically t-securely computes f with perfect security if for every real-world adversary, there exists an ideal-world adversary $\mathcal{S}$, such that for every $I \subseteq [n]$ of size at most t, it holds that*

$$\text{REAL}_{\pi,I,\mathcal{A}} \equiv \text{IDEAL}_{f,I,\mathcal{S}}.$$

Statistical security. We denote by κ the statistical security parameter. Let poly denote the set of all positive polynomials. A function $\nu\colon \mathbb{N} \to [0,1]$ is negligible if $\nu(\kappa) < 1/p(\kappa)$ for every $p \in \text{poly}$ and large enough κ. The statistical distance between two random variables X and Y over a finite set $\mathcal{U}$, denoted $\text{SD}(X,Y)$, is defined as,

$$\text{SD}(X,Y) \coloneqq \frac{1}{2} \cdot \sum_{u\in\mathcal{U}} \left|\Pr[X = u] - \Pr[Y = u]\right| .$$

Given two distribution ensembles $X = \{X(\kappa, a)\}_{\kappa\in\mathbb{N},a\in\{0,1\}^*}$ and $Y = \{Y(\kappa, a)\}_{\kappa\in\mathbb{N},a\in\{0,1\}^*}$, we denote $X \equiv_s Y$ if X and Y are statistically close, i.e., if $\mathrm{SD}(X, Y) \leq \nu(\kappa)$ for a negligible function ν.

Definition 3.4 (statistical security)**.** *Let $f : (\{0,1\}^*)^n \to (\{0,1\}^*)^n$ be an n-ary functionality. A protocol π statically t-securely computes f with statistical security if for every real-world adversary, there exists an ideal-world adversary $\mathcal{S}$, such that for every $I \subseteq [n]$ of size at most t, it holds that*

$$\mathrm{REAL}_{\pi,I,\mathcal{A}} \equiv_s \mathrm{IDEAL}_{f,I,\mathcal{S}}.$$

4 Separating Rushing from Super-Rushing

In this section, we show simple protocols that are secure against rushing adversaries but are insecure against super-rushing adversaries. We start by defining the $(2, 5)$-simultaneous-broadcast functionality, followed by two protocols that securely realize it against a rushing adversary corrupting one party but are insecure against a super-rushing adversary. Next, we show that as opposed to security against rushing adversaries that remains secure under sequential composition, security against super-rushing adversaries does not.

4.1 Rushing Security Does Not Imply Super-Rushing Security

$(2, 5)$ **simultaneous broadcast.** The functionality used in our separations is a variant of simultaneous broadcast [CGMA85], which involves five parties. Parties P_1 and P_2 hold input bits b_1 and b_2, respectively, whereas parties P_3, P_4, and P_5 have no input (formally, their input is the empty string λ). All parties learn the pair of bits (b_1, b_2). We emphasize that as opposed to naïve parallel composition of broadcast protocols, if P_i is corrupted for $i \in \{1, 2\}$, then its input must remain *independent* of the input of the other honest party P_{3-i}. That is, the functionality to compute is

$$f_{\mathsf{sb}}(b_1, b_2, \lambda, \lambda, \lambda) = (y, y, y, y, y) \qquad \text{where} \qquad y = (b_1, b_2).$$

4.1.1 A Protocol with No Committal Round

We proceed to present the protocol $\pi_{\mathsf{sb\text{-}basic}}$ (Protocol 4.1) that securely realizes f_{sb} in the presence of a rushing adversary but not in the presence of a super-rushing adversary. This is a basic protocol, described in [GIKR02], in which each of the parties P_1 and P_2 send their input bits to the non-input parties P_3, P_4, and P_5, who then echo these messages to everyone.

Protocol 4.1: $\pi_{\mathsf{sb\text{-}basic}}$ (a basic $(2, 5)$-simultaneous-broadcast protocol)

Round 1:

- For $i \in \{1, 2\}$, party P_i sends its input bit b_i to P_3, P_4, and P_5.
- For $j \in \{3, 4, 5\}$ and $i \in \{1, 2\}$, denote by b_i^j the bit P_j received from P_i; in case P_j did not receive a message from P_i, or if the message is not a bit, then P_j sets $b_i^j := 0$.

Round 2:

- For $j \in \{3, 4, 5\}$, party P_j sends the pair of bits (b_1^j, b_2^j) to all parties.
- For $k \in [5]$ and $j \in \{3, 4, 5\}$, denote by $(b_1^{j\to k}, b_2^{j\to k})$ the pair of bits P_k received from P_j; in case P_k did not receive a message from P_j, or if the message is not a pair of bits, then P_k sets $b_i^{j\to k} := 0$ for $i \in \{1, 2\}$.

Output:

- For $k \in [5]$ and $i \in \{1,2\}$, party P_k sets $c_i^k := \mathsf{maj}\{b_i^{3\to k}, b_i^{4\to k}, b_i^{5\to k}\}$.
- For $k \in [5]$, party P_k outputs (c_1^k, c_2^k).

We proceed to prove in Lemma 4.2 that the protocol is secure in the presence of a rushing adversary, and in Lemma 4.3 that the protocol is not secure in the presence of a super-rushing adversary.

Lemma 4.2. *Protocol $\pi_{\mathsf{sb\text{-}basic}}$ statically 1-securely realizes f_{sb} with perfect security in the presence of a malicious rushing adversary.*

Proof. We prove the theorem by considering two cases: the case where an input party is corrupted and when a non-input party is corrupted.

Corrupted input party. Let $\mathcal{A}$ be a malicious, rushing adversary that statically corrupts party P_i, for some $i \in \{1,2\}$. We construct a simulator $\mathcal{S}$, which upon receiving auxiliary input z proceeds as follows:

1. Invoke $\mathcal{A}$ with the auxiliary input z.
2. Receive from $\mathcal{A}$, the round-1 messages corresponding to the P_i, denoted as b_i^3, b_i^4, b_i^5, where b_i^j is the bit sent to party P_j; in case $\mathcal{A}$ does not deliver b_i^j or if b_i^j is not a bit, set $b_i^j := 0$.
3. Set $b_i = \mathsf{maj}\{b_i^3, b_i^4, b_i^5\}$ and send b_i to the trusted party as the input of the corrupted party P_i.
4. Receive from the trusted party the output $y = (b_1, b_2)$.
5. Send the round-2 messages from the honest parties P_3, P_4, and P_5; that is, for $j \in \{3,4,5\}$:
   - If $i = 1$, send the message (b_1^j, b_2) from P_j to P_i.
   - If $i = 2$, send the message (b_1, b_2^j) from P_j to P_i.
6. Output whatever $\mathcal{A}$ outputs and halt.

We proceed to prove that $\text{REAL}_{\pi_{\mathsf{sb\text{-}basic}},\{i\},\mathcal{A}} \equiv \text{IDEAL}_{f_{\mathsf{sb}},\{i\},\mathcal{S}}$ by describing a sequence of hybrids experiments. The output of each hybrid is the output of honest parties and of the simulator.

- HYB_1: The first hybrid experiment proceeds like the ideal world, but the simulator has full control over the trusted party computing f_{sb}: that is, the simulator can see all messages sent by honest parties and can determine their output. Here, the simulator simulates the real execution of all honest parties based on their inputs toward the adversary, and deliver their output according to this execution. Finally, the simulator outputs whatever $\mathcal{A}$ outputs. It is immediate that HYB_1 is identically distributed as $\text{REAL}_{\pi_{\mathsf{sb\text{-}basic}},\{i\},\mathcal{A}}$.

- HYB_2: The second hybrid experiment proceeds like HYB_1, but instead of sending the honest parties output according to the protocol, the simulator first receives the round-1 message from P_i, denoted b_i^3, b_i^4, b_i^5 (setting $b_i^j := 0$ if $\mathcal{A}$ does not deliver the relevant bit); next, the simulator sets $b_i := \mathsf{maj}\{b_i^3, b_i^4, b_i^5\}$, and send b_i to the trusted party as the input of the corrupted party P_i. The output of the honest parties is now computed by the trusted party.

  By the pigeonhole principle, the majority value of $\{b_i^3, b_i^4, b_i^5\}$ is well defined. Since P_3, P_4, and P_5 are honest and echo the bit they receive from P_i, and since honest parties in the protocol compute the output according to the majority, it follows that HYB_2 is identically distributed as HYB_1.

- HYB_3: The third hybrid experiment proceeds like HYB_2, but instead of obtaining the input of the second input-party P_{3-i} and sending the round-2 messages according to the protocol, the simulator uses the output value $y = (b_1, b_2)$ it receives from the trusted party after delivering the input of P_i, to extract the bit b_{3-i}. Next, the simulator simulates the round-2 message from the honest P_j to P_i, for $j \in \{3, 4, 5\}$, as follows:

  - If $i = 1$, send the message (b_1^j, b_2) from P_j to P_i.
  - If $i = 2$, send the message (b_1, b_2^j) from P_j to P_i.

  Since the output computed by trusted party $y = (b_1, b_2)$ contains both inputs bits, it follows that the second-round messages are identically distributed as in HYB_2; therefore, HYB_3 is identically distributed as HYB_2.

- HYB_4: The fourth hybrid experiment proceeds like HYB_3, but now the simulator cannot see the input sent by honest parties to the trusted party and cannot deliver their output. That is, HYB_4 is exactly the ideal computation of f_{sb}. Since in HYB_3 the simulator does not use its ability to see the honest inputs delivered to the trusted party, nor to set the honest outputs, it is immediate that HYB_4 is identically distributed as HYB_3; i.e., that $\text{IDEAL}_{f_{\mathsf{sb}},\{i\},\mathcal{S}}$ is identically distributed as HYB_3.

Corrupted non-input party. Let $\mathcal{A}$ be a malicious, rushing adversary that statically corrupts party P_j with $j \in \{3, 4, 5\}$. We construct a simulator $\mathcal{S}$, which upon receiving auxiliary input z proceeds as follows:

1. Invoke $\mathcal{A}$ with the auxiliary input z.
2. Send the empty string λ to the trusted party as the input of the corrupted P_j.
3. Receive from the trusted party the output $y = (b_1, b_2)$.
4. Send to $\mathcal{A}$ the round-1 messages corresponding to the P_j: the message b_1 from P_1 and the message b_2 from P_2.
5. Send to $\mathcal{A}$ the round-2 messages corresponding to the P_j: the message (b_1, b_2) from $P_{j'}$ with $j' \in \{3, 4, 5\} \setminus \{j\}$.
6. Receive the round-2 messages from the corrupted P_j: $(b_1^{j\to k}, b_2^{j\to k})$ for every $k \in [5] \setminus \{j\}$.
7. Output whatever $\mathcal{A}$ outputs and halt.

We proceed to prove that $\text{REAL}_{\pi_{\mathsf{sb\text{-}basic}},\{j\},\mathcal{A}} \equiv \text{IDEAL}_{f_{\mathsf{sb}},\{j\},\mathcal{S}}$ by describing a sequence of hybrids experiments. The output of each hybrid is the output of honest parties and of the simulator.

- HYB_1: The first hybrid experiment proceeds like the ideal world, but the simulator has full control over the trusted party computing f_{sb}: that is, the simulator can see all messages sent by honest parties and can determine their output. Here, the simulator simulates the real execution of all honest parties based on their inputs toward the adversary, and deliver their output according to this execution. Finally, the simulator outputs whatever $\mathcal{A}$ outputs. It is immediate that HYB_1 is identically distributed as $\text{REAL}_{\pi_{\mathsf{sb\text{-}basic}},\{j\},\mathcal{A}}$.

- HYB_2: The second hybrid experiment proceeds like HYB_1, but instead of sending the honest parties output according to the protocol, the simulator first sends the empty string λ to the trusted party as the input of the corrupted party P_j, and the honest parties' output is determined according to f_{sb}.

Since P_j does not have an input, it can only affect the output in the protocol using the round-2 messages $(b_1^{j\to k}, b_2^{j\to k})$ for every $k \in [5] \setminus \{j\}$. However, since the honest parties in the simulated protocol compute the majority according to the round-2 messages from P_3, P_4, and P_5, and the honest parties will send the same (b_1, b_2) according to the inputs of P_1 and P_2, it follows that HYB_2 is identically distributed as HYB_1.

- HYB_3: The third hybrid experiment proceeds like HYB_2, but instead of simulating the message toward P_j according to the protocol, the simulator uses the output $y = (b_1, b_2)$ it receives from the trusted party after sending the empty string λ as the input of the corrupted party P_j. That is, the simulator simulates receives the round-1 messages to P_j by sending b_1 from P_1 and b_2 from P_2, and sends the round-2 messages (b_1, b_2) from to P_j from $P_{j'}$ for $j' \in \{3, 4, 5\} \setminus \{j\}$.

  Since the output computed by the trusted party $y = (b_1, b_2)$ contains both inputs bits, it is immediate that these messages are identically distributed as in HYB_2. Hence, HYB_3 is identically distributed as HYB_2.

- HYB_4: The fourth hybrid experiment proceeds like HYB_3, but now the simulator cannot see the input sent by honest parties to the trusted party and cannot deliver their output. That is, HYB_4 is exactly the ideal computation of f_{sb}. Since in HYB_3 the simulator does not use its ability to see the honest inputs delivered to the trusted party, nor to set the honest outputs, it is immediate that HYB_4 is identically distributed as HYB_3; i.e., that $\text{IDEAL}_{f_{\mathsf{sb}},\{i\},\mathcal{S}}$ is identically distributed as HYB_3.

This completes the proof of Lemma 4.2. □

We proceed to show that $\pi_{\mathsf{sb\text{-}basic}}$ is vulnerable to attacks by a super-rushing adversary that cannot be simulated. Intuitively, a corrupted P_1 can send a round-1 message to P_3 and learn the input of P_2 before sending the first-round messages to P_4 and P_5. This breaks the independence of inputs in f_{sb}.

Lemma 4.3. *Protocol $\pi_{\mathsf{sb\text{-}basic}}$ does not 1-securely realize f_{sb} with perfect security in the presence of a super-rushing adversary.*

Proof. We start by constructing the following super-rushing adversary.

The super-rushing adversary $\mathcal{A}_{\mathsf{sr}}$. The adversary $\mathcal{A}_{\mathsf{sr}}$ corrupts P_1 and proceeds as follows:

1. Send 0 to the party P_3 as the message in round 1.
2. Upon receiving the round-2 message from P_3 of the form $(b_1^{3\to 1}, b_2^{3\to 1})$, send the round-1 message $b_2^{3\to 1}$ to P_4 and P_5.
3. Receive round-2 messages: $(b_1^{4\to 1}, b_2^{4\to 1})$ from P_4 and $(b_1^{5\to 1}, b_2^{5\to 1})$ from P_5, and halt.

The proof of the lemma follows from Claims 4.4 and 4.5 showing that for any simulator $\mathcal{S}$, the distributions of $\text{REAL}_{\pi_{\mathsf{sb\text{-}basic}},\{1\},\mathcal{A}_{\mathsf{sr}}}$ and $\text{IDEAL}_{f_{\mathsf{sb}},\{1\},\mathcal{S}}$ are not identical.

Claim 4.4. *Consider an execution of $\pi_{\mathsf{sb\text{-}basic}}$ on uniformly sampled inputs $b_1, b_2 \leftarrow \{0,1\}$ with the adversary $\mathcal{A}_{\mathsf{sr}}$ corrupting P_1. Denote by (y_1, y_2) the common output value of the honest parties. Define the event $\mathcal{E}_{\mathsf{real}}$ where $y_1 = y_2$. Then, $\Pr[\mathcal{E}_{\mathsf{real}}] = 1$, where the probability is over the choice of b_1 and b_2 (note that the protocol and the adversary are deterministic).*

Proof. The execution of $\pi_{\mathsf{sb\text{-}basic}}$ on (b_1, b_2) with $\mathcal{A}_{\mathsf{sr}}$ corrupting P_1 proceeds as follows:

- P_2 sends b_2 to P_3, P_4, and P_5.
- P_1 sends 0 to P_3.
- P_3 sends $(0, b_2)$ to P_1, P_2, P_4, and P_5.
- P_1 sends b_2 to P_4 and P_5.
- P_4 sends (b_2, b_2) to P_1, P_2, P_3, and P_5.
  Similarly, P_5 sends (b_2, b_2) to P_1, P_2, P_3, and P_4.
- For $k \in \{2, 3, 4, 5\}$, party P_k outputs $c_1 := \mathsf{maj}\{0, b_2, b_2\}$ and $c_2 := \mathsf{maj}\{b_2, b_2, b_2\}$, i.e., outputs (b_2, b_2).

It is always the case that $y_1 = y_2$, and thus $\Pr[\mathcal{E}_{\mathsf{real}}] = 1$ □

Claim 4.5. *Let $\mathcal{S}$ be a arbitrary simulator. Consider an ideal computation of f_{sb} on uniformly sampled inputs $b_1, b_2 \leftarrow \{0,1\}$ with the simulator $\mathcal{S}$ corrupting P_1. Denote by (y_1, y_2) the common output value of the honest parties. Define the event $\mathcal{E}_{\mathsf{ideal}}$ where $y_1 = y_2$. Then, $\Pr[\mathcal{E}_{\mathsf{ideal}}] = \frac{1}{2}$, where the probability is over the choice of b_1 and b_2 and the randomness of $\mathcal{S}$.*

Proof. The ideal computation of f_{sb} on (b_1, b_2) with $\mathcal{S}$ corrupting P_1 proceeds as follows:

- P_2 sends b_2 to the trusted party.
- P_1 sends a bit b to the trusted party.
- The trusted party sends (b, b_2) to all parties.

It follows that for every $b \in \{0, 1\}$ it holds that

$$\begin{aligned}\Pr[\mathcal{E}_{\mathsf{ideal}}] &= \Pr[b = b_2] \\ &= \Pr[b = b_2 \mid b_2 = 0] \cdot \Pr[b_2 = 0] + \Pr[b = b_2 \mid b_2 = 1] \cdot \Pr[b_2 = 1] \\ &= 0 \cdot \frac{1}{2} + 1 \cdot \frac{1}{2} = \frac{1}{2}.\end{aligned}$$

□

This concludes the proof of Lemma 4.3. □

4.1.2 A Protocol with a Committal Round but $\mathsf{ORR} = \mathsf{CR} + 1$

A closer look at the protocol $\pi_{\mathsf{sb\text{-}basic}}$ indicates that the protocol does not admit a committal round, i.e., a specific round CR such that the simulator can first simulate the protocol until round CR (including), next extract the corrupted party's input and send it to the trusted party to obtain the output, and then resume the simulation from round $\mathsf{CR} + 1$ till the end. Indeed, in case P_i with $i \in \{1, 2\}$ is corrupted, the simulator must extract the corrupted party's input in the first round, whereas if P_j with $j \in \{3, 4, 5\}$ is corrupted, the simulator must obtain the output to simulate the first round. One may wonder whether the source of the separation between rushing and super-rushing adversaries lies in the lack of a committal round.

Our second separation shows that this is not the case. That is, we show a protocol for $(2, 5)$ simultaneous broadcast, $\pi_{\mathsf{sb\text{-}vss}}$ (Protocol 4.6), that admits a committal round and is secure against a rushing adversary, yet is not secure against a super-rushing adversary. The protocol is based on the one-round verifiable secret sharing (VSS) from [GIKR01] for 5 parties with $t = 1$. The

one-round sharing phase is vanilla Shamir sharing—the dealer picks a polynomial of degree 1 (a line), and gives each party one point. The dealer does not participate in the reconstruction phase, and during reconstruction the parties (excluding the dealer) simply exchange their shares. That is, each party receives four shares, and can reconstruct if there is at most one error; in case there is no unique decoding (i.e., if three shares are co-linear, and one share does not lie on the same line), the parties output a default value.

In our protocol, P_1 and P_2 share their input in the first round, and the parties reconstruct the pair of bits in the second round. The protocol is secure against a rushing adversary, and the first round is a committal round. However, as before, a super-rushing adversary can advance a single sender to proceed to round 2 to obtain an additional share on the honest-party's polynomial. That is, it can recover the entire sharing polynomial of the honest party from $t + 1 = 2$ shares, and thereby, the input of the other honest party. Thus, a super-rushing adversary in an execution of the protocol $\pi_{\mathsf{sb\text{-}vss}}$ can violate input independence.

Protocol 4.6: $\pi_{\mathsf{sb\text{-}vss}}$ (VSS-based $(2,5)$-simultaneous-broadcast protocol)

The protocol is parameterized by a finite field $\mathbb{F}_p$ with $p > 5$.

Round 1:

- For $i \in \{1,2\}$, party P_i samples a polynomial $f_i \leftarrow \mathbb{F}_p[x]$ of degree at most 1 such that $f_i(0) = b_i$. Next, for every $k \in [5]$, party P_i sends to P_k the value $v_{i\to j} := f_i(j)$.
- For $j \in [5]$ and $i \in \{1,2\}$, denote by $v'_{i\to j}$ the message received by P_j from P_i; in case P_j did not receive a message from P_i, or if the message is not an element of $\mathbb{F}_p$, then P_j sets $v'_{i\to j} := 0$.

Round 2:

- Party P_1 sends $v'_{2\to 1}$ to all the parties. Party P_2 sends $v'_{1\to 2}$ to all the parties. For $j \in \{3,4,5\}$, party P_j sends $(v'_{1\to j}, v'_{2\to j})$ to all the parties.
- For $k \in [5]$, denote by $v_1^{j\to k}$ values P_k received from P_j (for $j \in \{2,3,4,5\}$) and by $v_2^{j\to k}$ values P_k received from P_j (for $j \in \{1,3,4,5\}$); in case P_k did not receive a message from P_j, or if the message is not an element of $\mathbb{F}_p$, then P_k sets $v_i^{j\to k} := 0$ for the corresponding $i \in \{1,2\}$.

Output:

- For $k \in [5]$ and $i \in \{1,2\}$, if $k = i$, set $f'_k(x) := f_i(x)$. If $k \neq i$, then party P_k interpolates a polynomial f'_i of degree at most 1 such that for at least 3 out of the 4 elements $j \in [5] \setminus \{k\}$ it holds that $f'_i(j) = v'_{1\to k}$; if P_k cannot interpolate such f'_i, set it to the zero polynomial.
- For $k \in [5]$, party P_k computes $c_1^k := f'_1(0)$ and $c_2^k := f'_2(0)$; if $c_i^k \notin \{0,1\}$, set $c_i^k := 0$. Finally, output (c_1^k, c_2^k).

The following lemma is proven similarly to Lemma 4.2, while relying on the fact that the VSS of [GIKR01] is secure in this setting; we omit the details.

Lemma 4.7. *Protocol $\pi_{\mathsf{sb\text{-}vss}}$ statically 1-securely realizes f_{sb} with perfect security in the presence of a malicious rushing adversary.*

We now show that there exists a super-rushing adversary that cannot be simulated.

Lemma 4.8. *Protocol $\pi_{\mathsf{sb\text{-}vss}}$ does not statically 1-securely realize f_{sb} with perfect security in the presence of a malicious super-rushing adversary.*

Proof. We show that a super-rushing adversary exists in the real world that breaks independence of inputs. That is, the adversary corrupts P_1 and chooses its input as a function of the input of P_2. Of course, such behavior cannot be simulated in the ideal world.

The super-rushing adversary. The super-rushing adversary $\mathcal{A}$ proceeds as follows:

- In the first round, the honest P_2 samples a random f_2 of degree at most 1 and gives each party P_i the share $s_i = f_2(i)$, for $i \in \{1, 3, 4, 5\}$.
- The corrupted P_1 receives a share s_1 from P_2 and sends to P_3 a random point v_3 in the field.
- Once P_3 receives all its round-1 messages—a share from P_1 and a share from P_2—it forwards the two shares to everyone; in particular, P_3 sends s_3 to P_1.
- The corrupted P_1 receives $s_3 = f_2(3)$, and together with $s_1 = f_2(1)$ (that it received directly from P_2) reconstructs $f_2(x)$ and learns $b_2 = f_2(0)$.
- Finally, P_1 sets $b_1 := b_2$, and interpolates a polynomial $f_1(x)$ of degree 1 such that $f_1(0) = b_2$ and $f_1(3) = v_3$. Next, P_1 continues running the protocol "honestly," providing parties with round-1 messages and round-2 messages, as described.

It is easy to see that in the real-world computation with $\mathcal{A}$, for every input b_2 for P_2, it always holds that all honest parties output (b_2, b_2). In the ideal-world computation, the simulator must choose the input b_1' for P_1 and send b_1' to the trusted party to obtain b_2. Therefore, for any choice of b_1' it holds that $\Pr[b_1' = b_2] = 1/2$, as b_1' must be chosen independently to b_2. □

4.2 Super-Rushing Security Is Not Maintained Under Sequential Composition

Sequential composition is a basic requirement of MPC protocols; it enables a simple form of a modular design in the following sense. Given an n-ary functionality f with a protocol π_f that securely realizes f, one can construct another protocol π in the f-hybrid model that securely realizes another function g by invoking the ideal computation of f multiple times, such that the invocations of f are done sequentially and each new invocation of f begins after the previous invocation has completed. The sequential composition operation ensures security of the protocol $\pi^{f \to \pi_f}$ that is obtained by replacing each invocation of f with an execution of π_f. The standard definition of MPC ensures sequential composition in the presence of a rushing adversary [Can00, Gol04].

In this section, we present a simple example highlighting that security is not maintained under sequential composition in the presence of a super-rushing adversary. The high-level idea is that a super-rushing adversary can learn information related to the second execution of π_f to influence the first execution. We note that the technique of [KLR06] of adding a dummy synchronization round between the executions enables sequential composition; however, as before, our focus is on unmodified protocols.[3]

[3]We note that a similar issue arises when sequentially composing protocols with probabilistic termination, i.e., protocols in which fast parties may complete the execution before slow parties do, and further a party does not know whether it is fast or slow [CCGZ19]. Such protocols naturally arise when using randomized Byzantine agreement as a subroutine in an MPC protocol. Sequential composition theorems in this setting indeed add a "local" dummy round in which a party does not communicate and ignores all incoming messages in this round to avoid super-rushing attacks [CCGZ19, CCGZ21].

4.3 The Hybrid Model and Sequential Composition

We begin by defining the hybrid model and stating the composition theorem for a rushing adversary. We refer the reader to [Can00] for further details.

The hybrid model. The *hybrid model* is a model that extends the real model with a trusted party that provides ideal computation for specific functionalities. The parties communicate with this trusted party in exactly the same way as in the ideal model described in Section 3.2.

Let f be an n-ary function; for implicity we will assume that f is deterministic. An execution of a protocol π computing a functionality g in the f-hybrid model involves the parties sending normal messages to each other (as in the real model) and in addition, having access to a trusted party computing f. It is essential that the invocations of f are done sequentially, meaning that before an invocation of f begins, the preceding invocation of f must finish and all parties hold their inputs, and the invocation ends once all honest parties obtain their outputs. In particular, there is at most one call to f per round, and no other messages are sent during any round in which f is called. Specifically, during a round in which the trusted party is called, the adversary in the hybrid model serves as an ideal-world adversary toward the trusted party. A round in which the trusted party is invoked proceeds as follows:

1. All of the inputs of the parties are determined at the onset of the round.
2. The parties hand their inputs to the trusted party. The values for the honest parties are determined by the protocol and the values for the corrupted party are determined by the adversary.
3. Once received all inputs, the trusted party computes the function f and sends to each party the corresponding output.

We denote by $\text{HYBRID}^{f}_{\pi,\mathcal{A}}(\boldsymbol{x}, z)$ the random variable consisting of the output of the honest parties and of the adversary following an execution of π in the f-hybrid model described above (with ideal calls to a trusted party computing f according to the ideal model) on input vector $\boldsymbol{x}$ and auxiliary input z given to $\mathcal{A}$. Let f be a deterministic n-ary function, let $\pi = (P_1, \ldots, P_n)$ be an n-party protocol in the f-hybrid model, and let $I \subseteq [n]$ denote the set of indices of the parties corrupted by $\mathcal{A}$. Denote by $\text{HYBRID}^{f}_{\pi,I,\mathcal{A}}(\boldsymbol{x}, z, \boldsymbol{\rho})$ the output vector of $P_1, \ldots, P_n$ and $\mathcal{A}$ resulting from the protocol interaction on input vector $\boldsymbol{x} = (x_1, \ldots, x_n)$, auxiliary input z, and randomness $\boldsymbol{\rho} = (\rho_{\mathcal{A}}, \rho_1, \ldots, \rho_n)$. Let $\text{HYBRID}^{f}_{\pi,I,\mathcal{A}}(\boldsymbol{x}, z)$ denote the probability distribution of $\text{HYBRID}^{f}_{\pi,I,\mathcal{A}}(\boldsymbol{x}, z, \boldsymbol{\rho})$ where $\boldsymbol{\rho}$ is uniformly chosen. Let $\text{HYBRID}^{f}_{\pi,I,\mathcal{A}}$ denote the distribution ensemble $\{\text{HYBRID}^{f}_{\pi,I,\mathcal{A}}(\boldsymbol{x}, z)\}_{(\boldsymbol{x},z)\in(\{0,1\}^*)^{n+1}}$.

Definition 4.9 (security in hybrid model)**.** *Let f and g be n-ary functions and suppose that f is deterministic. A protocol π statically t-securely computes g with perfect security in the f-hybrid model if for every adversary $\mathcal{A}$ for the f-hybrid model, there exists an ideal-world adversary $\mathcal{S}$, such that for every $I \subseteq [n]$ of size at most t, it holds that*

$$\text{HYBRID}^{f}_{\pi,I,\mathcal{A}} \equiv \text{IDEAL}_{g,I,\mathcal{S}}.$$

Sequential composition. The sequential composition theorem (see [Can00, Gol04]) guarantees the following.

Theorem 4.10. *Let f and g be a n-ary functions, and suppose that f is deterministic. Let π_f be a protocol that statically t-securely computes f with perfect security against a rushing adversary, and let π be a protocol that statically t-securely computes g with perfect security in the f-hybrid model against a rushing adversary. Denote by $\pi^{f\to\pi_f}$ the protocol that is obtained from π by replacing every call to f by an execution of π_f. Then, protocol $\pi^{f\to\pi_f}$ statically t-securely computes g with perfect security against a rushing adversary.*

4.3.1 Sequential Composition with a Super-Rushing Adversary

We begin by defining a 4-ary single-input function $f_{i\to\{j,k\}}$ and a protocol $\pi_{i\to\{j,k\}}$ that securely realizes it against super-rushing adversaries. Next, we define another 4-ary function g and a protocol π that securely realizes g in the $f_{i\to\{j,k\}}$-hybrid model by two sequential calls to $f_{i\to\{j,k\}}$. Finally, we show that the protocol obtained by replacing $f_{i\to\{j,k\}}$ with $\pi_{i\to\{j,k\}}$ is no longer secure against super-rushing adversaries.

The function $f_{i\to\{j,k\}}$ is a 4-ary function parameterized by three pairwise distinct indices $i, j, k \in [4]$. Party P_i has two input bits (x_1, x_2); the output of P_j is x_1 and the output of P_k is x_2. The two-round protocol $\pi_{i\to\{j,k\}}$ in which P_i sends x_1 to P_j in the first round and sends x_2 to P_k in the second round, is a simple protocol that realizes $f_{i\to\{j,k\}}$ against a super-rushing adversary in a trivial way for $t = 1$. As before, we can consider all-to-all communication by adding dummy messages.

Now consider the 4-ary function g in which defined as $g(x, \lambda, \lambda, (y_1, y_2)) = (y_1, y_2, x, \lambda)$; i.e., party P_1 outputs the first input bit of P_4, party P_2 outputs the second input bit of P_4, and party P_3 outputs the input bit of P_1.

Protocol 4.11: π: a protocol for the function g

Round 1: Party P_1 invokes $f_{1\to\{2,3\}}$ with input $(0, x)$.

Round 2: Party P_4 invokes $f_{4\to\{1,2\}}$ with input (y_1, y_2).

Output: Party P_1 outputs the bit it received in round 2, party P_2 outputs the bit it received in round 2, party P_3 outputs the bit it received in round 1, and party P_4 outputs nothing.

Claim 4.12. *The protocol π (Protocol 4.11) statically 1-securely computes g with perfect security in the $f_{i\to\{j,k\}}$-hybrid model in the presence of super-rushing adversary.*

Proof. We consider the case of each corrupted party separately. In each case, given an adversary $\mathcal{A}$ corrupting P_i we construct a simulator that interacts with the ideal computation of g and simulates the ideal computation of $f_{i\to\{j,k\}}$ to the adversary.

- For a corrupted P_1: The simulator waits to receive the message (b, x) from the adversary as the input for $f_{1\to\{2,3\}}$ in round 1, sends x to the ideal computation of g, obtains y as output, and simulates sending y as the output P_1 receives from the invocation of $f_{4\to\{1,2\}}$ in round 2.
- For a corrupted P_2: The simulator sends the empty string λ to the trusted party to obtain the output y. Next, $\mathcal{S}$ simulates sending 0 as the output P_2 receives from $f_{1\to\{2,3\}}$ in round 1, and simulates sending y as the output P_2 receives from $f_{4\to\{1,2\}}$ in round 2.
- For a corrupted P_3: The simulator sends the empty string λ to the trusted party to obtain the output x. Next, $\mathcal{S}$ simulates sending x as the output P_3 receives from $f_{1\to\{2,3\}}$ in round 1.

- For a corrupted P_4: The simulator waits to receive the message (y_1, y_2) from the adversary as the input for $f_{4\to\{1,2\}}$ in round 2, and sends (y_1, y_2) to the ideal computation of g.

Next, the simulator outputs whatever $\mathcal{A}$ outputs and halts. It is immediate to verify that the simulation is perfect in all cases. □

Claim 4.13. *The protocol $\pi^{f_{i\to\{j,k\}}\to\pi_{i\to\{j,k\}}}$ does not statically 1-securely computes g with perfect security in the presence of super-rushing adversary.*

Proof. First, observe that the protocol $\pi^{f_{i\to\{j,k\}}\to\pi_{i\to\{j,k\}}}$ proceeds as follows:

- **Round 1:** P_1 sends 0 to P_2.
- **Round 2:** P_1 sends x to P_3.
- **Round 3:** P_4 sends y_1 to P_1.
- **Round 4:** P_4 sends y_2 to P_2.

We define an adversary $\mathcal{A}$ that corrupts party P_1 and uses its super-rushing capabilities to ensure that P_3 outputs y_1 (the first input bit of P_4). The adversary proceeds honestly until round 2, where before sending a message to P_3, it first sends the dummy message to P_4 who advances to round 3 and sends the message y_1 to P_1. Next, $\mathcal{A}$ sends y_1 to P_3 and resumes the protocol until the end.

Now consider the following input distribution: $x, y_1 \leftarrow \{0,1\}$ are uniformly distributed, and $y_2 = y_1$. In an execution of $\pi^{f_{i\to\{j,k\}}\to\pi_{i\to\{j,k\}}}$ with $\mathcal{A}$ corrupting P_1 with this input distribution, it holds that both P_2 and P_3 output $x = y_2$ with probability 1. However, in the ideal computation of g, the adversary must choose the input x for the corrupted P_1 before learning the output. Since the input y_2 is uniformly distributed,the probability that $x = y_2$ is $1/2$. □

5 Single-Input Functionalities

In this section, we will consider functionalities in which only one party provides input, and will prove that protocols that securely realize such functionalities against *non-rushing* adversaries with perfect security, remain secure against super-rushing adversaries.

As a motivating example, consider a VSS functionality in which a dealer, say P_1, inputs a bivariate polynomial $S(x, y)$ of degree t in both variables. The functionality verifies that S is of degree t, and if so, each party P_i outputs $S(x, i)$ and $S(i, y)$. Otherwise, each party outputs $\perp$. This is the formalism of a VSS given, e.g., in [AAY21]. Another motivating example is the generation of multiplication triplets with a dealer, in which the dealer inputs three polynomials A, B, and C, the functionality verifies that all polynomials are of degree t and that $C(0) = A(0) \cdot B(0)$, and if so gives to each party P_i the output $(A(i), B(i), C(i))$.

Typically, the security of protocols for single-input functionalities is shown by constructing a simulator under two cases: when the input provider is corrupted and when the input provider is honest. We show the former in Section 5.1 and the latter in Section 5.2.

5.1 Corrupted Input Provider

We first consider the case where the input provider is corrupted. We begin by defining a *compatible* simulation. That is, where the simulator simulates the honest parties towards the adversary using uniformly distributed random coins (recall that the honest parties do not have inputs in this

case). Next, the simulator extracts the effective input for the corrupted input provider and outputs whatever the adversary outputs.

While this simulation structure is standard in many security proofs, it will be too strong for our setting and we will need to weaken it slightly. The reason is that given a super-rushing adversary $\mathcal{A}_{\mathsf{sr}}$, we will construct a non-rushing adversary $\mathcal{A}_{\mathsf{nr}}$, that will guess the random coins of all honest parties and will run in its head an execution with $\mathcal{A}_{\mathsf{sr}}$. Next, $\mathcal{A}_{\mathsf{nr}}$ will interact with the protocol according to the transcript in its virtual execution with $\mathcal{A}_{\mathsf{sr}}$; in case $\mathcal{A}_{\mathsf{nr}}$ did not guess the random coins correctly (i.e., if the messages in the protocol are different than in its virtual execution with $\mathcal{A}_{\mathsf{sr}}$) then $\mathcal{A}_{\mathsf{nr}}$ will abort and output $\perp$. It will suffice for our proof to be able to simulate $\mathcal{A}_{\mathsf{nr}}$ with a compatible simulation only when $\mathcal{A}_{\mathsf{nr}}$ does not abort.

Definition 5.1 (compatible simulation)**.** *Let f be an n-ary single-input functionality and let π be an n-party protocol. We say that π statically t-securely computes f with perfect security in the presence of non-rushing adversaries with a* compatible simulation*, if for every non-rushing adversary $\mathcal{A}$ there exists a "compatible" ideal-world adversary $\mathcal{S}$ such that for every $I \subseteq [n]$ of size at most t (which includes the input provider), every auxiliary input z, and every initial input x for the corrupted input provider, the following holds:*

Denote by $\mathcal{E}_{\mathsf{no\text{-}abort}}$ the event that the non-rushing adversary $\mathcal{A}$ does not abort. Then, either $\Pr[\mathcal{E}_{\mathsf{no\text{-}abort}}] = 0$, or it holds that

$$\text{IDEAL}_{f,I,\mathcal{S}}(x,z) \equiv \text{REAL}_{\pi,I,\mathcal{A}}(x,z) \mid \mathcal{E}_{\mathsf{no\text{-}abort}} \ .$$

Further, $\mathcal{S}$ operates according to Structure 5.2.

Structure 5.2: Compatible simulator $\mathcal{S}(z, I, x; (\rho_j)_{j\notin I})$ for a corrupted input provider

The simulator $\mathcal{S}$ operates on auxiliary input z, on input x for the corrupt input provider, and by corrupting the set of parties indexed by I. The simulator $\mathcal{S}$ will simulate the honest parties, denoted as $(\widetilde{P}_j)_{j\notin I}$, towards $\mathcal{A}$. The random tape of $\mathcal{S}$ consists of the random tapes used for simulating the honest parties, i.e., $(\rho_j)_{j\notin I}$.

1. **Initialization:** For every $j \notin I$, the simulator $\mathcal{S}$ initializes the simulated honest party $\widetilde{P}_j$ with ρ_j.
2. **Simulation:** $\mathcal{S}$ interacts with $\mathcal{A}(z, I, x)$, in a round-by-round fashion, by simulating the virtual honest parties $(\widetilde{P}_j)_{j\notin I}$ towards $\mathcal{A}$.
3. **Extraction:** Once the simulated execution completes, $\mathcal{S}$ computes some x', sends it to the trusted party, outputs whatever $\mathcal{A}$ outputs, and halts.

Theorem 5.3. *Let f be a single-input n-ary functionality and let π be an n-party protocol. Assume that π statically t-securely computes f with perfect security a compatible simulation in the presence of* non-rushing *adversaries with that corrupt the input provider.*

Then, π statically t-securely computes f with perfect security in the presence super-rushing *adversaries that corrupt the input provider.*

Proof. Let $\mathcal{A}_{\mathsf{sr}}$ be a super-rushing adversary; without loss of generality, assume that $\mathcal{A}_{\mathsf{sr}}$ is deterministic. We begin by constructing a non-rushing adversary $\mathcal{A}_{\mathsf{nr}}$ from $\mathcal{A}_{\mathsf{sr}}$.

The non-rushing adversary $\mathcal{A}_{\mathsf{nr}}$. The adversary $\mathcal{A}_{\mathsf{nr}}$ receives the auxiliary input z, the set of corrupted parties I, and the corrupted input-provider's input $x \in \{0,1\}^*$. For every $j \notin I$, we will explicitly denote the random tape used by $\mathcal{A}_{\mathsf{nr}}$ for the virtual honest party by ρ_j.

1. For every $j \notin I$, initialize the virtual honest party $\widetilde{P}_j$ with no input but with random tape ρ_j.
2. Denote by R the total number of rounds of π; for every $r \in [R]$, initialize the set $M_r := \emptyset$ (that will be used to store messages sent between $\mathcal{A}_{\mathsf{nr}}$ to $\mathcal{A}_{\mathsf{sr}}$ for round r).
3. Invoke $\mathcal{A}_{\mathsf{sr}}(z, I, x)$.
4. **Send messages to $\mathcal{A}_{\mathsf{sr}}$:** Whenever $\mathcal{A}_{\mathsf{sr}}$ asks to receive a message (j^*, i^*, r) for $j^* \notin I$ and $i^* \in I$ proceed as follows:
   (a) If there exists a message $(j^*, i^*, r, m^r_{j^* \to i^*}) \in M_r$, respond with $m^r_{j^* \to i^*}$.
   (b) Otherwise, verify that for every $i \in I$, a message of the form $(i, j^*, r-1, \cdot)$ exists in M_{r-1} (i.e., $\mathcal{A}_{\mathsf{sr}}$ provided all messages from corrupted parties to P_{j^*} in round $r-1$; for $r = 1$ this holds vacuously). If this does not hold, call this event $\mathsf{notSuperRushing}$ and abort.[4]
   (c) Invoke the next-message function of the virtual honest party $\widetilde{P}_{j^*}$ on its random coins and incoming messages until (and including) round $r-1$. Add all the messages to M_r.
   (d) Respond to $\mathcal{A}_{\mathsf{sr}}$ with the message $(j^*, i^*, r, m^r_{j^* \to i^*})$.
5. **Receive messages from $\mathcal{A}_{\mathsf{sr}}$:** Whenever $\mathcal{A}_{\mathsf{sr}}$ sends a message $m^r_{i \to j}$ (a message from a corrupted P_i to an honest P_j in round r), add the tuple $(i, j, r, m^r_{i \to j})$ to M_r.
6. **Interact with π:** Once all sets $M_1, \ldots, M_R$ are determined, interact with the honest parties in π as follows. For each round $r \in [R]$:
   (a) For every $j \notin I$ and every $i \in I$, send the message $m^r_{i \to j}$ from P_i to P_j according to M_r.
   (b) For every $j \notin I$ and every $i \in I$, receive message $m^r_{j \to i}$ from the honest P_j to the corrupted P_i. If $m^r_{j \to i}$ is not as expected in M_r, then abort.

By the security of π, there exists a compatible simulator $\mathcal{S}_{\mathsf{nr}}$ that can perfectly simulate $\mathcal{A}_{\mathsf{nr}}$ conditioned on $\mathcal{A}_{\mathsf{nr}}$ not aborting. We will now construct a simulator $\mathcal{S}_{\mathsf{sr}}$ and for the super-rushing adversary $\mathcal{A}_{\mathsf{sr}}$ using $\mathcal{S}_{\mathsf{nr}}$.

The super-rushing simulator $\mathcal{S}_{\mathsf{sr}}$. Consider the compatible non-rushing simulator $\mathcal{S}_{\mathsf{nr}}$ for $\mathcal{A}_{\mathsf{nr}}$ that is guaranteed to exist by Definition 5.1. Upon receiving (z, I, x), the super-rushing simulator $\mathcal{S}_{\mathsf{sr}}$ chooses random ρ_j for every $j \notin I$ uniformly at random, and invokes $\mathcal{S}_{\mathsf{nr}}(z, I, x; (\rho_j)_{j \notin I})$. Once $\mathcal{S}_{\mathsf{nr}}$ outputs the view of $\mathcal{A}_{\mathsf{nr}}$ together with an input value x', the super-rushing simulator $\mathcal{S}_{\mathsf{sr}}$ sends x' to the trusted party, outputs view, and halts.

We now proceed to prove that for every z, every I (that includes the input provider), and every x, it holds that

$$\mathrm{REAL}_{\pi, I, \mathcal{A}_{\mathsf{sr}}}(x, z) \equiv \mathrm{IDEAL}_{f, I, \mathcal{S}_{\mathsf{sr}}}(x, z) \ .$$

We start by claiming that in an execution of π, the non-rushing adversary $\mathcal{A}_{\mathsf{nr}}$ does not abort in Step 6b with non-zero probability.

Claim 5.4. *Consider an execution of π with $\mathcal{A}_{\mathsf{nr}}$ corrupting parties in I over uniformly distributed random coins for the honest parties. Then,*

$$\Pr\left[\mathcal{A}_{\mathsf{nr}} \textit{ does not abort}\right] > 0 \ .$$

[4] Note that since $\mathcal{A}_{\mathsf{nr}}$ interacts with $\mathcal{A}_{\mathsf{sr}}$, $\mathsf{notSuperRushing}$ never occurs and we include this event for completeness.

Proof. To that end, observe that $\mathcal{A}_{\mathsf{nr}}$ invokes $\mathcal{A}_{\mathsf{sr}}$ in its head and samples the random tapes for the virtual honest parties to simulate an execution with $\mathcal{A}_{\mathsf{sr}}$ and populate the sets $M_1, \ldots, M_R$ with messages from the virtual honest parties. If the random tapes sampled by $\mathcal{A}_{\mathsf{nr}}$ for the virtual honest parties are exactly the random tapes of the honest parties in the execution of π with $\mathcal{A}_{\mathsf{nr}}$, then $\mathcal{A}_{\mathsf{nr}}$ receives messages from honest parties to the corrupted parties that match the messages in the sets $M_1, \ldots, M_R$. Denote by $\tilde{\rho}_J = (\tilde{\rho}_j)_{j \notin I}$ the random tapes of the real virtual honest parties and $\rho_J = (\rho_j)_{j \notin I}$ the random tapes of the real honest parties in an execution of π with the adversary $\mathcal{A}_{\mathsf{nr}}$. Hence, we have that

$$\Pr\left[\mathcal{A}_{\mathsf{nr}} \text{ does not abort}\right] \geq \Pr\left[\tilde{\rho}_J = \rho_J\right] > 0 \ . \qquad \square$$

Claim 5.5. $\text{IDEAL}_{f,I,\mathcal{S}_{\mathsf{sr}}}(x,z) \equiv \text{IDEAL}_{f,I,\mathcal{S}_{\mathsf{nr}}}(x,z)$.

Proof. Observe that $\mathcal{S}_{\mathsf{sr}}$ simply executes $\mathcal{S}_{\mathsf{nr}}$ with uniformly random tapes for the simulated honest parties, send to the trusted party whatever $\mathcal{S}_{\mathsf{nr}}$ sends to the trusted party, and outputs whatever $\mathcal{S}_{\mathsf{nr}}$ outputs. Therefore, this is just a syntactic change and the above equation holds. $\square$

Claim 5.6. $\text{IDEAL}_{f,I,\mathcal{S}_{\mathsf{nr}}}(x,z) \equiv \text{REAL}_{\pi,I\mathcal{A}_{\mathsf{nr}}}(x,z) \mid \mathcal{A}_{\mathsf{nr}}$ *does not abort.*

Proof. First, observe that by Claim 5.4 the distribution on the right hand side is well-defined as the probability that $\mathcal{A}_{\mathsf{nr}}$ does not abort is non-zero. Conditioned on not aborting, $\mathcal{A}_{\mathsf{nr}}$ receives in the real execution of π the same messages that were used in its internal simulation with $\mathcal{A}_{\mathsf{sr}}$.

In the ideal computation with $\mathcal{S}_{\mathsf{nr}}$, the adversary $\mathcal{A}_{\mathsf{nr}}$ is invoked with the same randomness that is used by the simulated honest parties, which guarantees that $\mathcal{A}_{\mathsf{nr}}$ does not abort. Hence, from the security of π against $\mathcal{A}_{\mathsf{nr}}$ with compatible simulator $\mathcal{S}_{\mathsf{nr}}$, we have that the above equation holds. $\square$

Claim 5.7. $\text{REAL}_{\pi,I,\mathcal{A}_{\mathsf{nr}}}(x,z) \mid \mathcal{A}_{\mathsf{nr}}$ *does not abort* $\equiv \text{REAL}_{\pi,I,\mathcal{A}_{\mathsf{sr}}}(x,z)$.

Proof. Again, by Claim 5.4, the distribution on the left hand side is well-defined as the probability that $\mathcal{A}_{\mathsf{nr}}$ does not abort is non-zero. By construction, conditioned on not aborting, $\mathcal{A}_{\mathsf{nr}}$ identically simulates an execution of π towards the super-rushing $\mathcal{A}_{\mathsf{sr}}$, and interacts with the honest parties accordingly. Therefore, the output of the real computation in this case is identically distributed as the output when interacting with $\mathcal{A}_{\mathsf{sr}}$. $\square$

The proof of Theorem 5.3 now follows from Claims 5.5 to 5.7. $\square$

5.2 Honest Input Provider

For the case of an honest input provider, the corrupted parties hold no input and the simulator must simulate the adversary's view given only the outputs of the corrupted parties. We first present the requirement from the underlying simulation.

Definition 5.8 (compatible simulation)**.** *Let f be an n-ary single-input functionality and let π be an n-party protocol. We say that π statically t-securely computes f with perfect security in the presence of non-rushing adversaries with a* compatible simulation*, if for every non-rushing adversary $\mathcal{A}$ there exists a "compatible" ideal-world adversary $\mathcal{S}$ such that for every $I \subseteq [n]$ of size at most t (which does not include the input provider), every auxiliary input z, and every input of the honest input provider, the following holds:*

Denote by $\mathcal{E}_{\mathsf{no\text{-}abort}}$ the event that the non-rushing adversary $\mathcal{A}$ does not abort. Then, either $\Pr\left[\mathcal{E}_{\mathsf{no\text{-}abort}}\right] = 0$, or it holds that

$$\text{IDEAL}_{f,I,\mathcal{S}}(x,z) \equiv \text{REAL}_{\pi,I,\mathcal{A}}(x,z) \mid \mathcal{E}_{\mathsf{no\text{-}abort}} \ .$$

Further, $\mathcal{S}$ operates according to Structure 5.9.

Structure 5.9: Compatible simulator $\mathcal{S}(z, I, (y_i)_{i\in I}; (\rho_j)_{j\notin I})$ for an honest input provider.

The simulator $\mathcal{S}$ operates on auxiliary input z, on output values $(y_i)_{i\in I}$, and by corrupting the set of parties indexed by I. The simulator $\mathcal{S}$ will simulate the honest parties, denoted as $(\widetilde{P}_j)_{j\notin I}$, towards $\mathcal{A}$. The random tape of $\mathcal{S}$ consists of the random tapes used for simulating the honest parties, i.e., $(\rho_j)_{j\notin I}$.

1. **Initialization:** For every $j \notin I$, the simulator $\mathcal{S}$ initializes the simulated honest party $\widetilde{P}_j$ with ρ_j. Further, $\mathcal{S}$ computes a value x' such that $f_I(x') = (y_i)_{i\in I}$, where $f_I(\cdot)$ denotes the outputs of the parties in I.[5] Set the input value for the simulated input provider to be x'.
2. **Simulation:** $\mathcal{S}$ interacts with $\mathcal{A}(z, I)$, in a round-by-round fashion, by simulating the virtual honest parties $(\widetilde{P}_j)_{j\notin I}$ towards $\mathcal{A}$.
3. **Output:** Once the simulated execution completes, $\mathcal{S}$ outputs whatever $\mathcal{A}$ outputs, and halts.

Theorem 5.10. *Let f be a single-input functionality and let π be an n-party protocol. Assume that π statically t-securely computes f with perfect security with compatible simulation in the presence of* non-rushing *adversaries that do not corrupt the input provider.*

Then, π statically t-securely computes f with perfect security in the presence super-rushing *adversaries that do not corrupt the input provider.*

Proof. Let $\mathcal{A}_{\mathsf{sr}}$ be a super-rushing adversary; without loss of generality, assume that $\mathcal{A}_{\mathsf{sr}}$ is deterministic. We begin by constructing a non-rushing adversary $\mathcal{A}_{\mathsf{nr}}$ from $\mathcal{A}_{\mathsf{sr}}$. The main difference from the case of a corrupted input-provider, is that in addition to sampling the randomness of the virtual honest parties, $\mathcal{A}_{\mathsf{nr}}$ also samples a uniformly random input x on behalf of the (virtual) honest input-provider and simulates an execution with $\mathcal{A}_{\mathsf{sr}}$ with the input x.

The non-rushing adversary $\mathcal{A}_{\mathsf{nr}}$. The adversary $\mathcal{A}_{\mathsf{nr}}$ receives the auxiliary input z and the set of corrupted parties I. For every $j \notin I$, we will explicitly denote the random tape used by $\mathcal{A}_{\mathsf{nr}}$ for the virtual honest party by ρ_j.

1. Initialize virtual honest parties $\widetilde{P}_j$ with random tapes ρ_j for every $j \notin I$ and only the honest input-provider holding an input, x' which is sampled uniformly at random.
2. Initialize $M_1, \ldots, M_R \leftarrow \emptyset$.
3. Invoke $\mathcal{A}_{\mathsf{sr}}(z, I)$.
4. **Send messages to $\mathcal{A}_{\mathsf{sr}}$:** Whenever $\mathcal{A}_{\mathsf{sr}}$ asks to receive a message (j^*, i^*, r) for $j^* \notin I$ and $i^* \in I$:
   (a) If there exists a message $(j^*, i^*, r, m^r_{j^*\to i^*}) \in M_r$ then give $m^r_{j^*,i^*}$.
   (b) If there is no message $(j^*, i^*, r, m^r_{j^*\to i^*})$ then verify that all messages $(i, j^*, r-1)$ are in M_{r-1} for all $i \in I$ (i.e., the adversary provided all the messages from the corrupted parties to P_{j^*} in round $r-1$; when $r = 1$ this holds vacuously). If this does not hold, call this event $\mathsf{notSuperRushing}$[6] and abort.

[5] Note that this step may not be efficient; in case efficient simulation is of interest, the simulator can be parameterized by x'.

[6] Note that since $\mathcal{A}_{\mathsf{nr}}$ interacts with $\mathcal{A}_{\mathsf{sr}}$, $\mathsf{notSuperRushing}$ never occurs and we include this event as a sanity check.

(c) Invoke the next-message function of the virtual honest party P_{j^*} on all its incoming messages at round $r-1$. Add all the messages to M_r.

(d) Give to $\mathcal{A}_{\mathsf{sr}}$ the message $(j^*, i^*, r, m^r_{j^* \to i^*})$.

5. **Receive messages from $\mathcal{A}_{\mathsf{sr}}$:** Whenever $\mathcal{A}_{\mathsf{sr}}$ sends a message $m^r_{i \to j}$ – a message from corrupted P_i to honest P_j in round r, add the quadruple $(i, j, r, m^r_{i \to j})$ to M_r.
6. **Outgoing interaction:** Once all sets $M_1, \ldots, M_r$ are determined, interact with the honest parties as follows. In each round $r = 1, \ldots, R$:
   (a) Output messages $m^r_{i \to j}$ for every $j \notin I$ and $i \in I$ according to M_r.
   (b) Receive messages $m^r_{j \to i}$ for every $j \notin I$ and $i \in I$.
   (c) If $m^r_{j \to i}$ are not as expected in M_r then abort. Else, proceed to the next round.

We now construct a simulator $\mathcal{S}_{\mathsf{sr}}$ and show that it can simulate any super-rushing adversary $\mathcal{A}_{\mathsf{sr}}$ that does not corrupt the input provider.

The super-rushing simulator $\mathcal{S}_{\mathsf{sr}}(z, I, (y_i)_{i \in I}; (\rho_j)_{j \notin I})$. We now build the super-rushing simulator $\mathcal{S}_{\mathsf{sr}}$. Consider the compatible rushing simulator $\mathcal{S}_{\mathsf{nr}}$ as per Structure 5.9.

1. The simulator $\mathcal{S}_{\mathsf{sr}}$ first receives from the trusted party, the outputs of the corrupted parties denoted as $(y_i)_{i \in I}$.
2. $\mathcal{S}_{\mathsf{sr}}$ chooses random ρ_j for every $j \notin I$ and executes $\mathcal{S}_{\mathsf{nr}}(z, I, (y_i)_{i \in I}; (\rho_j)_{j \notin I})$. That is, it fixes the random tape $(\rho_j)_{j \notin I}$ for the simulated honest parties in the execution of $\mathcal{S}_{\mathsf{nr}}$ and forwards the corrupted parties' outputs $(y_i)_{i \in I}$ to $\mathcal{S}_{\mathsf{nr}}$.
3. At the end of its execution, $\mathcal{S}_{\mathsf{nr}}$ outputs the view view of $\mathcal{A}_{\mathsf{nr}}$. $\mathcal{S}_{\mathsf{sr}}$ outputs view and halts.

We now proceed to prove that for every z, every I (that does not include the input provider), and every x, it holds that

$$\text{REAL}_{\pi, I, \mathcal{A}_{\mathsf{sr}}}(x, z) \equiv \text{IDEAL}_{f, I, \mathcal{S}_{\mathsf{sr}}}(x, z) \ .$$

Note that the honest input provider holds x as input. We start by showing that in an execution of π, the non-rushing adversary $\mathcal{A}_{\mathsf{nr}}$ does not abort in Step 6c with non-zero probability.

Claim 5.11. *Consider an execution of π with $\mathcal{A}_{\mathsf{nr}}$ corrupting parties in I over uniformly distributed random coins for the honest parties. Let x denote the honest input provider's input. Then,*

$$\Pr\left[\mathcal{A}_{\mathsf{nr}} \textit{ does not abort}\right] > 0 \ .$$

Proof. To that end, observe that $\mathcal{A}_{\mathsf{nr}}$ samples the random tapes of the virtual honest parties in its execution and populates the sets $M_1, \ldots, M_R$ with messages from the virtual honest parties. Furthermore, $\mathcal{A}_{\mathsf{nr}}$ also samples the honest input provider's input uniformly at random as x'. If the random tapes sampled by $\mathcal{A}_{\mathsf{nr}}$ for the virtual honest parties are exactly the random tapes of the honest parties in the execution of π with $\mathcal{A}_{\mathsf{nr}}$ **and** if $x' = x$ (where x is the honest input provider's input in the execution of π), then $\mathcal{A}_{\mathsf{nr}}$ receives messages from honest parties to the corrupted parties that match the messages in the sets $M_1, \ldots, M_R$. Denote by $\tilde{\rho}_J = (\tilde{\rho}_j)_{j \notin I}$ the random tapes of the virtual honest parties and $\rho_J = (\rho_j)_{j \notin I}$ the random tapes of the honest parties in an execution of π with the adversary $\mathcal{A}_{\mathsf{nr}}$. Hence, we have that

$$\Pr\left[\mathcal{A}_{\mathsf{nr}} \text{ does not abort}\right] \geq \Pr\left[\tilde{\rho}_J = \rho_J \wedge x' = x\right] > 0 \ . \qquad \square$$

Claim 5.12. $\text{IDEAL}_{f, I, \mathcal{S}_{\mathsf{sr}}}(x, z) \equiv \text{IDEAL}_{f, I, \mathcal{S}_{\mathsf{nr}}}(x, z)$.

Proof. Observe that $\mathcal{S}_{\mathsf{sr}}$ just executes $\mathcal{S}_{\mathsf{nr}}$ with uniformly random tapes for the simulated honest parties and invokes $\mathcal{S}_{\mathsf{nr}}$ with the corrupted parties' outputs $(y_i)_{i\in I}$. Furthermore, $\mathcal{S}_{\mathsf{sr}}$ outputs whatever $\mathcal{S}_{\mathsf{nr}}$ outputs and the trusted party is invoked with the same input x in both executions. This is just a syntactic change and the above equation holds. □

Claim 5.13. $\text{IDEAL}_{f,I,\mathcal{S}_{\mathsf{nr}}}(x,z) \equiv \text{REAL}_{\pi,I,\mathcal{A}_{\mathsf{nr}}}(x,z) \mid \mathcal{A}_{\mathsf{nr}}$ *does not abort.*

Proof. First, observe that by Claim 5.11, the distribution on the right hand side is well-defined as the probability that $\mathcal{A}_{\mathsf{nr}}$ does not abort is non-zero. Conditioned on not aborting, $\mathcal{A}_{\mathsf{nr}}$ receives in the real execution of π the same messages that were used in its internal simulation with $\mathcal{A}_{\mathsf{sr}}$.

In the ideal computation with $\mathcal{S}_{\mathsf{nr}}$, the adversary $\mathcal{A}_{\mathsf{nr}}$ is invoked with the same randomness that is used by the simulated honest parties, which guarantees that $\mathcal{A}_{\mathsf{nr}}$ does not abort. Furthermore, observe that the honest input provider holds the same input in both executions. Hence, from the security of π against $\mathcal{A}_{\mathsf{nr}}$ with compatible simulator $\mathcal{S}_{\mathsf{nr}}$, we have that the above equation holds. □

Claim 5.14. $\text{REAL}_{\pi,I,\mathcal{A}_{\mathsf{nr}}}(x,z) \mid \mathcal{A}_{\mathsf{nr}}$ *does not abort* $\equiv \text{REAL}_{\pi,I,\mathcal{A}_{\mathsf{sr}}}(x,z)$.

Proof. Once again, by Claim 5.11, the distribution on the left is well-defined as the probability that $\mathcal{A}_{\mathsf{nr}}$ does not abort is non-zero. By construction, conditioned on not aborting, $\mathcal{A}_{\mathsf{nr}}$ identically simulates an execution of π towards the super-rushing $\mathcal{A}_{\mathsf{sr}}$, and interacts with the honest parties accordingly. Therefore, the output of the real computation in this case is identically distributed as the output when interacting with $\mathcal{A}_{\mathsf{sr}}$. □

The proof of Theorem 5.10 now follows from Claims 5.12 to 5.14. □

6 Sufficient Conditions for Perfect Security with Super-Rushing Adversary

In this section, we show sufficient conditions under which security in the presence of a rushing adversary implies security in the presence of a super-rushing adversary. Our theorem considers protocols that satisfy some requirements, such as the existence of a committal round. Moreover, it assumes that the protocol can be proven secure against a rushing adversary using a simulator the follows some specific structure. We start with the definitions of the requirements from the simulation.

6.1 Compatible Simulation

Our requirements from the simulation of the rushing adversary are as follows:

Definition 6.1. *A simulator $\mathcal{S}(z, I, (x_i)_{i\in I}; \rho_{\mathcal{S}})$ for a protocol π simulating a rushing adversary $\mathcal{A}$, where z is the auxiliary input, I is the set of corrupted parties, x_i is P_i's original input for each $i \in I$, and $\rho_{\mathcal{S}}$ is the randomness of $\mathcal{S}$, parameterized by a set $\boldsymbol{\beta} = (\beta_j)_{j\notin I}$ from some domain B, is called* compatible *if it satisfies the following properties:*

- **Simulation via virtual honest parties:** *$\mathcal{S}$ computes the view of the adversary by simulating virtual honest parties with default inputs, and follows the structure as described in Structure 6.2.*

- **Simulation for every default inputs:** *The simulation works for every set of default input $(\beta_j)_{j\notin I}$ in some domain B over the possible inputs of the function. That is, we denote by $\mathcal{S}[\boldsymbol{\beta}]$ the simulator that uses $\boldsymbol{\beta} = (\beta_j)_{j\notin I}$ as default inputs. That is, for every real-world rushing adversary $\mathcal{A}$, and for every $\boldsymbol{\beta}$ it holds that:*

$$\left\{\text{REAL}_{\pi,\mathcal{A}(z),I}(\boldsymbol{x})\right\}_{\boldsymbol{x}\in(\{0,1\}^*)^n} \equiv \left\{\text{IDEAL}_{f,\mathcal{S}[\boldsymbol{\beta}](z),I}(\boldsymbol{x})\right\}_{\boldsymbol{x}\in(\{0,1\}^*)^n}$$

Structure 6.2: Structure of simulation via virtual honest parties

1. **Initialization:** $\mathcal{S}$ interacts with the real-world adversary $\mathcal{A}(z, I, (x_i)_{i\in I})$ as an oracle, in a round-by-round fashion. $\mathcal{S}$ initializes simulated honest parties denoted as $(\tilde{P}_j)_{j\notin I}$, and initializes each $\tilde{P}_j$ with default input β_j.
2. **Simulation for rounds $r = 1, \ldots, \mathsf{CR}$:** In each round $r \leq \mathsf{CR}$ the simulator sends to $\mathcal{A}$ all the messages from the honest parties to the corrupted parties, each is computed according to the next-message function of the protocol π on the default input and the simulated incoming messages. The adversary $\mathcal{A}$ then replies with all messages $\{m^r_{i\to j}\}_{j\notin I, i\in I}$ where $m^r_{i\to j}$ is the message from corrupted P_i to an honest $\tilde{P}_j$ in round r.
3. **At the end of round CR:** The simulator sends to the trusted party the input values of the corrupted parties, $\{x'_i\}_{i\in I}$.
4. **Simulation for rounds $r = \mathsf{CR}+1, \ldots, \mathsf{ORR}-1$:** Proceeds as in the simulation of rounds $r = 1, \ldots, \mathsf{CR}$.
5. **At the end of round $\mathsf{ORR}-1$:** The simulator receives from the trusted party the output values of the corrupted parties, $\{y_i\}_{i\in I}$.
6. **Simulator for round $r = \mathsf{ORR}$:** Let $J = [n] \setminus I$. For every party $j^* \notin I$, there exists a function SimOut_{j^*} such that:

$$m^{\mathsf{ORR}}_{j^*\to I} = \mathsf{SimOut}_{j^*}\left((y_i)_{i\in I},\quad m^{[\mathsf{ORR}-1]}_{J\to J},\quad m^{[\mathsf{ORR}-1]}_{I\to j^*}\right),$$

   where for $A, B \subseteq [n]$, and $R \subseteq [1, \ldots, \mathsf{ORR}]$, $m^R_{A,B} = (m^r_{a,b})_{a\in A, b\in B, r\in R}$. That is, the messages from P_{j^*} to the corrupted parties in the final round is a function of the outputs of the corrupted parties, the honest-to-honest communication in all rounds up to $\mathsf{ORR}-1$, and the messages P_{j^*} received from the corrupted parties up to round $\mathsf{ORR}-1$.
7. The simulator outputs whatever $\mathcal{A}$ outputs, without loss of generality, its view.

Intuitively, we assume that the simulator works by simulating honest parties with some default inputs (and that the simulation can work with any set of default inputs, as standard in secret-sharing-based simulations), and then at the ORR the simulator can "fix" the results. We highlight one important aspect of SimOut_{j^*}—the function to generate the messages that honest P_{j^*} sends in the last round—this function is based on all honest parties' communication up to round $\mathsf{ORR}-1$, on the outputs of the corrupted parties, and on all the messages P_{j^*} received in round $\mathsf{ORR}-1$; however, the function is *independent* of the messages other honest parties received in round $\mathsf{ORR}-1$. This clearly holds in the real protocol (as incoming messages that honest parties receive in round r will be propagated in outgoing messages in round $r+1$), and we require that it would also hold in the simulation.

Broadcast. For simplicity and to ease the presentation, we present the simulation where the parties use only point-to-point communication and no broadcast. For protocols with broadcast, we remark that SimOut should also obtain all the messages that P_{j^*} received over its point-to-point channels and over the broadcast channel.

Simulating after round ORR. For simplicity, we assume that the round where the output is revealed, that is, round ORR is the final round of the protocol. Note that for certain protocols this may not be the case and there may be multiple rounds after round ORR. However, we note that our strategy for simulating round ORR can be directly applied for any round $r \geq \mathsf{ORR}$ if required. Critically, the messages of each honest party in any round $r \geq \mathsf{ORR}$ must only depend on the honest party's incoming traffic and the outputs of the corrupted parties.

6.2 The Main Theorem

We show the following theorem:

Theorem 6.3. *Let f be an n-ary functionality and let π be an n-party protocol with all-to-all communication. Assume that:*

- *π t-securely realizes f with perfect security against rushing adversaries (as per Definition 3.3);*
- *π admits a committal round CR, an output-revealing round ORR, and it holds that $\mathsf{ORR} > \mathsf{CR} + 1$;*
- *The simulator $\mathcal{S}$ is compatible as per Definition 6.1.*

Then, π t-securely realizes f with perfect security against super rushing adversaries.

6.3 Examples

Before proving Theorem 6.3, we first demonstrate that several well-known secret-sharing-based protocols from the literature meet the conditions stated in Theorem 6.3. We assume the reader has some familiarity with those protocols.

Protocols based on secret-sharing. We capture a wide variety of protocols that are based on secret-sharing such as [BGW88, CCD88, BB89, GRR98, CDM00, ALR11, AAY21]. We consider the BGW protocol [BGW88] and its proof of security [AL17]. The general simulation strategy we describe below also holds for (1) protocols that make use of packed secret sharing [FY92, CCXY18]; (2) non-threshold secret sharing (for security against general adversary structures) [HM97, HM00]; and (3) protocols that are based on multiplication triplets with offline-online phases [Bea91, HMP00, HM01, BH08, CP17, AAPP23].

The BGW protocol. We consider the BGW protocol [BGW88] and its proof of security [AL17]. The simulator described in [AL17, Theorem 7.2] simulates the protocol by considering virtual honest parties, assigning them default inputs of 0 (of course, this is arbitrary), and then executing the protocol honestly using those inputs. The committal round is the end of the VSS round, in which the simulator extracts the inputs of the corrupted parties.

VSS. Before proceeding, we provide further details on the VSS. We claim that the shared polynomial can be reconstructed by analyzing the all-to-all communication and the information transmitted over the broadcast channel. To see this, recall that to share a secret s, the dealer selects a bivariate polynomial $S(x, y)$ of degree t in both variables such that $S(0, 0) = s$. The dealer then sends each party P_j a pair of univariate polynomials, $(f_j(x), g_j(y)) = (S(x, j), S(j, y))$. The parties perform verification steps, during which each party P_k receives sub-shares $f_j(k)$ and $g_j(k)$. Finally, After some additional verification steps, the parties vote over the broadcast channel on whether to accept or reject the shares.

If the dealer is honest, the parties will always accept, and from the shares sent to all honest parties, the polynomial $S(x, y)$ of the dealer and the secret s can be determined. Similarly, if the dealer is corrupted and the shares are accepted, there must be at least $t + 1$ honest parties that accepted, and their sub-shares can be used to reconstruct the univariate shares, and then the bivariate shares. The shared polynomial can be reconstructed by inspecting the all-to-all communication and the information transmitted over the broadcast channel.

The circuit emulation. The protocol maintains an invariant where the value of each wire in the emulated circuit is hidden by a random degree-t polynomial $F(x)$, with each party P_j holding the share $F(j)$. Similar to VSS, other sub-protocols of BGW, such as multiplication, adhere to this structure, where honest-to-honest communication (along with information on the broadcast channel) is sufficient to reconstruct the polynomials that hide the wire values, as well as the shares held by each party (and in fact, the entire adversary's view). It is important to note that in the simulation, the constant term of all internal wires is incorrect.

Simulating the output wires. Assume for simplicity that $|I| = t$. When reaching an output wire that is associated with some corrupted P_{i^*}, the simulator can use honest-to-honest communication to reconstruct the polynomial $F_{i^*}(x)$ that hides that output wire. The adversary already knows $F_{i^*}(i)$ for every $i \in I$, i.e., it knows t points on a degree-t polynomial, which still does not fully determine the polynomial. Moreover, the constant term of $F_{i^*}(x)$ in the simulation is incorrect.

At the point, the simulator "corrects" the polynomial by interpolating another degree-t polynomial $\widetilde{F}_{i^*}(x)$ such that $\widetilde{F}_{i^*}(i) = F_{i^*}(i)$ for every $i \in I$, and $\widetilde{F}_{i^*}(0) = y_i$, where y_i is the true output on that wire. Those $t + 1$ points fully determine this polynomial. Each honest party P_j then sends to P_{i^*} the shares $\widetilde{F}_{i^*}(j)$. The functions SimOut_{j^*} are defined along those lines, while the important property is that the "invariant" of the simulation can be reconstructed from the honest-to-honest communication up until round $\mathsf{ORR} - 1$.

6.4 Proof of Theorem 6.3

Proof. Fix a super-rushing adversary $\mathcal{A}'$. We construct a simulator $\mathcal{S}'$. The simulator is described as two processes that are run in parallel:

1. The interaction with the super-rushing adversary $\mathcal{A}'$; The simulator sends and receives messages to that adversary.
2. The interaction with the rushing simulator $\mathcal{S}$. The simulator $\mathcal{S}'$ (together with $\mathcal{A}'$) behaves as a rushing adversary when interacting with $\mathcal{S}$. Since we know exactly how $\mathcal{S}$ simulates the view of the adversary, we do not use it to generate messages; rather, we use it to extract the effective inputs from $\mathcal{A}'$.

The super-rushing simulator $\mathcal{S}'(z, I, (x_i)_{i\in I})$. We now construct the simulator $\mathcal{S}'$ for the super-rushing adversary $\mathcal{A}'$. The simulator receives as input the auxiliary input z, the set of corrupted parties I, and the initial inputs of the corrupted parties. The simulator proceeds as follows:

1. Initialize $M_1, \ldots, M_{\mathsf{ORR}} \leftarrow \emptyset$, and output values $(y_i)_{i\in I} = \perp$.
2. Sample randomness $\rho_{\mathcal{S}}$ for the invocations of the underlying simulator $\mathcal{S}$, and invoke the simulator $\mathcal{S}(z, I, (x_i)_{i\in I}; \rho_{\mathcal{S}})$.
3. Invoke the super-rushing adversary $\mathcal{A}'(z, I, (x_i)_{i\in I})$.
4. $\mathcal{S}'$ invokes the simulated honest parties $\widetilde{P}_j$ for every $j \notin I$ with some default inputs. Extract the randomness for each simulator honest $\widetilde{P}_j$ from $\rho_{\mathcal{S}}$ in a similar way as $\mathcal{S}$ works.
5. **Interaction with $\mathcal{A}'$ for every** $r = 1, \ldots, \mathsf{ORR}$: Send and receive messages from $\mathcal{A}'$ as follows:
   Send messages to $\mathcal{A}'$: Whenever the adversary $\mathcal{A}'$ asks to receive the message (j^*, i^*, r) for $j^* \notin I$ and $i^* \in I$:
   (a) If there exists a message $(j^*, i^*, r, m^r_{j^*\to i^*}) \in M_r$, then give $m^r_{j^*,i^*}$ to $\mathcal{A}'$ as the result and break.
   (b) If there is no message $(j^*, i^*, r, m^r_{j^*\to i^*})$ in M_r, then verify that all messages $(i, j^*, r-1)$ are in M_{r-1} for all $i \in I$ (i.e., the adversary provided all the messages from the corrupted parties to P_{j^*} in round $r-1$; when $r = 1$ this holds vacuously) and all messages $(j, j^*, r-1)$ are also in M_{r-1} (again, for $r = 1$ this holds vacuously). If this does not hold, call this event $\mathsf{notSuperRushing}$ and abort.
   (c) If $r = 1, \ldots, \mathsf{ORR} - 1$ and there is no message $(j^*, i^*, r, m^r_{j^*\to i^*})$ in M_r: Invoke the next-message function of the virtual honest party P_{j^*} on all its incoming messages at round $r-1$. Add all the messages to M_r.
   (d) If $r = \mathsf{ORR}$ and there is no message $(j^*, i^*, r, m^r_{j^*\to i^*})$ in M_r:
      i. Verify that $(y_i)_{i\in I} \neq \perp$ are stored (see Step 6b). If not, call this event $\mathsf{noOutputs}$ and abort.
      ii. Verify that all the messages from honest parties to honest parties are stored in all $M_1, \ldots, M_{\mathsf{ORR}-1}$. If not, call this event $\mathsf{notReadyForORR}$ and abort.
      iii. Compute:
      $$m^{\mathsf{ORR}}_{j^*\to I} = \mathsf{SimOut}_{j^*}\left((y_i)_{i\in I}, \quad m^{[\mathsf{ORR}-1]}_{J,J}, \quad m^{[\mathsf{ORR}-1]}_{I,j^*}\right).$$
      Add those messages to M_{ORR}.
   (e) Give to $\mathcal{A}'$ the message $(j^*, i^*, r, m^r_{j^*\to i^*})$ that was just generated.

   Receive messages from the adversary: Whenever $\mathcal{A}'$ sends a message $m^r_{i,j}$ (a message from corrupted P_i to honest P_j in round r add the tuple $(i, j, r, m^r_{i,j})$ to M_r.
6. **Interaction with $\mathcal{S}$ for every** $r = 1, \ldots, \mathsf{ORR}$**:**
   (a) Once all the messages $(k, \ell, r, \cdot) \in M_r$ for all $k \in [n]$ and $\ell \in [n]$:
      i. Receive from $\mathcal{S}$ all the messages from the honest parties to the corrupted parties in round r. Verify that all the received messages are exactly the same in M_r. Otherwise, call this event $\mathsf{notConsistent}$, and abort.
      ii. Output all the messages from the corrupted parties to the honest parties that are in M_r.
   (b) In round $\mathsf{CR}+1$: In addition to the messages from the honest parties to the corrupted parties for round $\mathsf{CR}+1$, the simulator $\mathcal{S}$ also outputs the extracted inputs $(x'_i)_{i\in I}$. Send $(x'_i)_{i\in I}$ to the trusted party as the effective inputs of the corrupted parties, and receive back $(y_i)_{i\in I}$. Store those values.

7. Output whatever $\mathcal{A}'$ outputs, and halt.

We now show that the simulator is valid. We have the following claims.

Claim 6.4. *The super-rushing simulator $\mathcal{S}'$ never aborts due to* noSuperRushing *event in Step 5b.*

Proof. Assuming that $\mathcal{A}'$ is a valid super-rushing adversary, the simulator $\mathcal{S}'$ never aborts in Step 5b. Since $\mathcal{A}'$ is super rushing, whenever it asks to see a message (j^*, i^*, r) it is guaranteed that all messages to P_{j^*} from corrupted parties in round $r-1$ were already provided by $\mathcal{A}'$. □

Claim 6.5. *The super-rushing simulator $\mathcal{S}'$ never aborts due to* noOutputs *event in Step 5(d)i.*

Proof. Consider the stage of the execution when $\mathcal{S}'$ executes Step 5(d)i for the first time. Suppose that the adversary has requested to receive the message (j^*, i^*, ORR). Since $\mathcal{A}'$ is super-rushing, it must have already provided the corrupted parties' messages to this honest party P_{j^*} corresponding to round $\mathsf{ORR}-1$. Furthermore, as all parties talk to all parties, $\mathcal{A}'$ must have already provided all the corrupted parties' messages to **all** honest parties until round $\mathsf{ORR}-2$ (including round $\mathsf{ORR}-2$). Since $\mathsf{ORR} > \mathsf{CR}+1$, it must hold that $\mathsf{ORR}-2 \geq \mathsf{CR}$. Hence, $\mathcal{A}'$ must have provided all the corrupted parties' messages to the honest parties until at least round CR. That is, the underlying rushing simulator $\mathcal{S}$ has already executed simulation until at least the committal round CR. Hence, $\mathcal{S}'$ must have already executed Step 6b for round $\mathsf{CR}+1$ and stored the outputs $(y_i)_{i\in I}$. Henceforth, this must hold every time $\mathcal{S}'$ executes Step 5(d)i. Hence, $\mathcal{S}'$ does not abort due to the noOutputs event in Step 5(d)i. □

Claim 6.6. *The super-rushing simulator $\mathcal{S}'$ never aborts due to* notReadyForORR *event in Step 5(d)ii.*

Proof. Consider the stage of the execution when $\mathcal{S}'$ executes Step 5(d)ii for the first time. Suppose that the adversary has requested to receive the message (j^*, i^*, ORR). Since $\mathcal{A}'$ is super-rushing, it must have provided the messages of the corrupted parties to the honest parties up to round $\mathsf{ORR}-1$. Furthermore, each virtual honest party has been invoked (for at least one message to P_{j^*}) until round $\mathsf{ORR}-1$. Hence, $\mathcal{S}'$ must have executed the next-message function of each of the honest parties until round $\mathsf{ORR}-1$ and stored all the round r messages in M_r for $r \leq \mathsf{ORR}-1$. As a result, when Step 5(d)ii is executed for the first time, $\mathcal{S}'$ will not abort. Since it holds at the first execution of Step 5(d)ii and messages are not deleted from any set M_r, we have that $\mathcal{S}'$ never aborts due to the notReadyForORR event. □

Claim 6.7. *The super-rushing simulator $\mathcal{S}'$ never aborts due to* notConsistent *event in Step 6(a)i.*

Proof. We separate between rounds $r = 1, \ldots, \mathsf{ORR}-1$ and round $r = \mathsf{ORR}$:

Up to round $\mathsf{ORR}-1$: The simulator $\mathcal{S}'$ fixes the random tape of $\rho_{\mathcal{S}}$ and treats it as a "white box", and, in particular, computes all the messages that are sent to $\mathcal{A}$ exactly the same as $\mathcal{S}$ does. Specifically, for each message (j^*, i^*, r), both simulators run the virtual honest party P_{j^*} on all round $r-1$ messages and the same random tape. Therefore, both will receive the same message $m^r_{j^*\to i^*}$.

Round ORR**:** Consider a message (j^*, i^*, ORR). This message was computed in Step 5(d)iii via an invocation of SimOut_{j^*} on

1. The outputs of the corrupted parties $(y_i)_{i \in I}$. These are guaranteed to be stored by $\mathcal{S}'$ due to Claim 6.5.
2. All the messages from honest to honest parties up to round $\mathsf{ORR} - 1$, otherwise, $\mathsf{notReadyForORR}$ occurs.
3. All the messages from corrupted parties to P_{j^*} in round $\mathsf{ORR} - 1$, otherwise, $\mathsf{notSuperRushing}$ occurs.

Internally, the simulator $\mathcal{S}$ also invokes SimOut_{j^*} on exactly the same inputs and with its random tape $\rho_{\mathcal{S}}$ fixed by $\mathcal{S}'$. Therefore, we have that for each (j^*, i^*, ORR) the message inserted by $\mathcal{S}'$ into M_{ORR} and the message output by $\mathcal{S}$ are identical, $m^{\mathsf{ORR}}_{j^* \to i^*}$.

From those two cases, we conclude that $\mathcal{S}'$ never aborts due to the $\mathsf{notConsistent}$ event in Step 6(a)i. □

The simulator $\mathcal{S}'$ perfectly simulates $\mathcal{A}'$. We now show that the outputs of the real and ideal are identically distributed. We prove this via the following hybrid experiments:

1. HYB_0: This is the ideal execution; The output of this execution is the view of the adversary $\mathcal{A}'$ and the output of the honest parties in the ideal world.
2. HYB_1: In this hybrid experiment, the simulator $\mathcal{S}'$ receives from the trusted party the inputs of the honest parties $(x_j)_{j \notin I}$ as input. Then, we modify both $\mathcal{S}'$ and $\mathcal{S}$, and use the real inputs $(x_j)_{j \notin I}$ instead of default inputs for the simulator honest parties $\widetilde{P}_j$ for every $j \notin I$.
3. HYB_2: We change the generation of the messages $m^{\mathsf{ORR}}_{j \to i}$ for every $j \notin I$ and $i \in I$ inside $\mathcal{S}'$ and $\mathcal{S}$. Instead of generating them according to the simulator using SimOut, we generate each $m_{j \to i}$ using the next-message function of P_j on all its incoming messages in round $\mathsf{ORR} - 1$ (and its private input and private state).
4. HYB_3: We change the outputs of the honest parties. Instead of using the outputs as generated by the trusted party, we use the outputs as generated by the simulated honest parties executed by $\mathcal{S}$.
5. HYB_4: This is the real execution: The output is the view of the adversary $\mathcal{A}'$ and the outputs of the honest parties.

We now show that every pair of consecutive hybrids, as defined in the proof of Theorem 6.3, is identically distributed. For convenience, we will use $\text{HYB}(\mathcal{S}^{\mathcal{A}})$ to denote the execution of a particular hybrid with the simulator $\mathcal{S}$ with the adversary $\mathcal{A}$.

Before we show that every two consecutive hybrids are identically distributed, we construct a rushing adversary $\mathcal{A}$, given a super-rushing adversary $\mathcal{A}'$. Looking ahead, we invoke this adversary with appropriate rushing simulators in the hybrid experiments to show that two consecutive hybrids are identically distributed.

Super-Rushing $\implies$ Rushing. Fix the super-rushing adversary $\mathcal{A}'$. From $\mathcal{A}'$ we construct a rushing adversary $\mathcal{A}$ (see Adversary 6.8) that invokes $\mathcal{A}'$ as an oracle.

Adversary 6.8: $\mathcal{A}^{\mathcal{A}'}\left(z, I, (x_i)_{i \in I}; (\rho_j)_{j \notin I}, (x'_j)_{j \notin I}\right)$

Input. The adversary $\mathcal{A}$ receives the auxiliary input z, the set of corrupted parties I and the initial inputs of the corrupted parties $(x_i)_{i\in I}$. We make $\mathcal{A}$'s internal choice of the input x'_j and the random tape ρ_j for each virtual honest party $\widetilde{P}_j$ explicit.

Execution.

1. Initialize each virtual honest party $\widetilde{P}_j$ with input x'_j and random tape ρ_j for every $j \notin I$.
2. Initialize $M_1, \ldots, M_R \leftarrow \emptyset$.
3. Invoke $\mathcal{A}'(z, I, (x_i)_{i\in I})$.
4. **Send messages to $\mathcal{A}'$:** Whenever $\mathcal{A}'$ asks to receive a message (j^*, i^*, r) for $j^* \notin I$ and $i^* \in I$:
   (a) If there exists a message $(j^*, i^*, r, m^r_{j^*\to i^*}) \in M_r$ then give $m^r_{j^*,i^*}$.
   (b) If there is no message $(j^*, i^*, r, m^r_{j^*\to i^*})$ then verify that all messages $(i, j^*, r-1)$ are in M_{r-1} for all $i \in I$ (i.e., the adversary provided all the messages from the corrupted parties to P_{j^*} in round $r-1$; when $r = 1$ this holds vacuously). If this does not hold, call this event notSuperRushing and abort.
   (c) Invoke the next-message function of the virtual honest party P_{j^*} on all its incoming messages at round $r-1$. Add all the messages to M_r.
   (d) Give to $\mathcal{A}'$ the message $(j^*, i^*, r, m^r_{j^*\to i^*})$.
5. **Receive messages from $\mathcal{A}'$:** Whenever $\mathcal{A}'$ sends a message $m^r_{i,j}$ – a message from corrupted P_i to honest P_j in round r, add the quadruple $(i, j, r, m^r_{i\to j})$ to M_r.
6. **Outgoing interaction:** Once all sets $M_1, \ldots, M_r$ are determined, interact with the honest parties as follows. In each round $r = 1, \ldots, R$:
   (a) Receive messages $m^r_{j\to i}$ for every $j \notin I$ and $i \in I$. If $m^r_{j\to i}$ are not as expected in M_r, then abort.
   (b) Output messages $m^r_{i\to j}$ for every $j \notin I$ and $i \in I$ according to M_r, and proceed to the next round.

We now show that every two consecutive hybrids are identically distributed:

Claim 6.9. HYB_0 *and* HYB_1 *are identically distributed.*

Proof. As per Claim 6.7, the messages generated by $\mathcal{S}'$ and by $\mathcal{S}$ are consistent, and remain the same in the modified hybrid (as it changes the way they are generated in both), and therefore we can just focus on $\mathcal{S}$. As in Definition 6.1 – the simulation of $\mathcal{S}$ is perfect for any possible default inputs, and in particular holds also for the true inputs $(x_j)_{j\notin I}$. □

Claim 6.10. HYB_1 *and* HYB_2 *are identically distributed.*

Proof. In HYB_1 the messages $m^{\mathsf{ORR}}_{j\to i}$ are generated according to SimOut, whereas in HYB_2 those are generated by the next-message function of the protocol, while using the real inputs $(x_j)_{j\notin I}$ as inputs. Observe that $\mathcal{S}'$ and $\mathcal{S}$ provide the exact same outputs.

Consider the simulator $\mathcal{S}'$ in both hybrids and denote by $\mathcal{S}'_1(z, I, \boldsymbol{x}; (\rho_j)_{j\notin I})$ the simulator in HYB_1, and by $\mathcal{S}'_2(z, I, \boldsymbol{x}; (\rho)_{j\notin I})$ the simulator in HYB_2. Note that $\mathcal{S}'_1$ and $\mathcal{S}'_2$ receive as input the actual inputs of the honest parties $(x_j)_{j\notin I}$ and we make explicit the random tape of each of the simulated honest parties as ρ_j for each $j \notin I$. Suppose that $\mathcal{S}'_1$ and $\mathcal{S}'_2$ invoke the super-rushing adversary $\mathcal{A}'(z, I, (x_i)_{i\in I})$.

Let $\mathcal{S}_1(z, I, \boldsymbol{x}; (\rho_j)_{j \notin I})$ and $\mathcal{S}_2(z, I, \boldsymbol{x}; (\rho_j)_{j \notin I})$ be the compatible simulators that invoke the rushing adversary $\mathcal{A}^{\mathcal{A}'}$ (see Adversary 6.8) in HYB_1 and HYB_2, respectively. Observe that we make explicit the random tape ρ_j of each simulated honest party $\widetilde{P}_j$ for $j \notin I$ in the execution of $\mathcal{S}_1$ and $\mathcal{S}_2$. We complete the proof with the following steps:

1. For each $\rho_J = (\rho_j)_{j \notin I}$, we claim that

$$\text{HYB}_1\left(\mathcal{S}_1'^{\mathcal{A}'}(\rho_J)\right) \equiv \text{HYB}_1\left(\mathcal{S}_1^{\mathcal{A}(\rho_J, (x_j)_{j \notin I})}(\rho_J)\right) ,$$

   where $\mathcal{S}_1'$ with random tape ρ_J for the simulated honest parties, invokes $\mathcal{S}_1$ and $\mathcal{A}$ with the same random tapes ρ_J for the honest parties. In addition, $\mathcal{A}$ is invoked with the random inputs $(x'_j)_{j \notin I}$ set as the actual inputs $(x_j)_{j \notin I}$ of the honest parties.
   Observe that $\mathcal{S}_1'$ executes exactly the same steps as $\mathcal{S}_1$ invoked with $\mathcal{A}$. The random tapes and the inputs for the simulated honest parties are identical in the execution of $\mathcal{S}_1'$ and the execution of $\mathcal{S}_1$ with $\mathcal{A}$. Additionally, the messages sent by $\mathcal{A}$ are identical to the messages sent by $\mathcal{A}'$. These changes are syntactic and hence the view of $\mathcal{A}'$ in the first execution is identical to the view of $\mathcal{A}$ in the second execution. Furthermore, $\mathcal{S}_1'$ uses $\mathcal{S}_1$ to extract the inputs $(x'_i)_{i \in I}$ of the corrupted parties after round $\mathsf{CR} + 1$. The trusted party computes $(y_1, \ldots, y_n) = f(x'_1, \ldots, x'_n)$ where $x'_j = x_j$ is the actual input of each honest party P_j. Both $\mathcal{S}_1'$ and $\mathcal{S}_1$ receive $(y_i)_{i \in I}$ and use SimOut to simulate the messages in round ORR (note that from Claim 6.5, we have that $\mathcal{S}_1'$ has already performed the input extraction and stored the outputs $(y_i)_{i \in I}$). Also, the trusted party computes f on the same inputs in both hybrid executions we consider.
   All the above are syntactic changes and as a result, on randomness ρ_J, the hybrid execution with $\mathcal{S}_1'$ is identical to the hybrid execution with $\mathcal{S}_1$ with $\mathcal{A}$.
2. For each ρ_J, we also claim that

$$\text{HYB}_2\left(\mathcal{S}_2'^{\mathcal{A}'}(\rho_J)\right) \equiv \text{HYB}_2\left(\mathcal{S}_1^{\mathcal{A}(\rho_J, (x_j)_{j \notin I})}(\rho_J)\right) ,$$

   where $\mathcal{S}_2'$ with random tape ρ_J for the simulated honest parties, invokes $\mathcal{S}_2$ and $\mathcal{A}$ with the same random tapes ρ_J for the honest parties. In addition, $\mathcal{A}$ is invoked with the random inputs $(x'_j)_{j \notin I}$ set as the actual inputs $(x_j)_{j \notin I}$ of the honest parties.
   This case is similar to the execution with $\mathcal{S}_1', \mathcal{S}_1$ and $\mathcal{A}$ with one distinction: $\mathcal{S}_2'$ (and $\mathcal{S}_2$) simulates the round ORR messages using the next-message function of the simulated honest parties. Since $\mathcal{A}'$ is valid super-rushing adversary, it has already provided the messages of the corrupted parties to each of the honest parties that need to be simulated in round ORR. Hence, $\mathcal{S}_2'$ can invoke the next-message function of each of the simulated honest parties to simulate round ORR. $\mathcal{S}_2$ can also do the same as the adversary $\mathcal{A}$ is rushing and has provided all round $\mathsf{ORR} - 1$ messages in this stage of the execution (see Claim 6.4). Once again the changes are syntactic and as a result, on randomness ρ_J, the hybrid execution with $\mathcal{S}_2'$ is identical to the hybrid execution with $\mathcal{S}_2$ with $\mathcal{A}$.
3. Now, we claim for each ρ_J, that

$$\text{HYB}_1\left(\mathcal{S}_1^{\mathcal{A}(\rho_J, (x_j)_{j \notin I})}(\rho_J)\right) \equiv \left\{\text{REAL}_{\pi(\rho_J, (x_j)_{j \notin I}), \mathcal{A}(\rho_J, (x_j)_{j \notin I}), I}(\boldsymbol{x})\right\} .$$

   In the hybrid execution, $\mathcal{S}_1$ is invoked with $\mathcal{A}$ with the same random tape ρ_j for each honest party P_j. Furthermore, $\mathcal{A}$ is invoked with the random inputs $(x'_j)_{j \notin I}$ set as the actual inputs $(x_j)_{j \notin I}$ of the honest parties that are received by $\mathcal{S}_1$ as input. In the real execution, π is executed with each honest party P_j for $j \notin I$ holding input x_j and random tape ρ_j. The

adversary $\mathcal{A}$ is invoked in the real execution with the same random tape ρ_J and the correct inputs $(x_j)_{j\notin I}$ of the honest parties in the real execution.
Let $\mathcal{S}$ be a compatible rushing simulator for π (see Definition 6.1). The simulator $\mathcal{S}_1$ is similar to $\mathcal{S}$ for π such that: $\mathcal{S}_1$ receives as input everything $\mathcal{S}$ receives as input and in addition receives the actual inputs of the honest parties. From the perfect security of π with compatible simulation, we can conclude that the hybrid execution with $\mathcal{S}$ is identical to the hybrid execution with $\mathcal{S}_1$ and hence, the hybrid execution with $\mathcal{S}_1$ is identical to the real execution with $\mathcal{A}$.

4. Finally, we claim for each ρ_J, that

$$\text{HYB}_2\left(\mathcal{S}_2^{\mathcal{A}(\rho_J,(x_j)_{j\notin I})}(\rho_J)\right) \equiv \left\{\text{REAL}_{\pi(\rho_J,(x_j)_{j\notin I}),\mathcal{A}(\rho_J,(x_j)_{j\notin I}),I}(\boldsymbol{x})\right\} \ .$$

In the hybrid execution, $\mathcal{S}_2$ is invoked with $\mathcal{A}$ with the same random tape ρ_J for the honest parties. Furthermore, $\mathcal{A}$ is invoked with the random inputs $(x'_j)_{j\notin I}$ set as the actual inputs $(x_j)_{j\notin I}$ of the honest parties that are received by $\mathcal{S}_2$ as input. In the real execution, π is executed with each honest party P_j for $j \notin I$ holding input x_j and random tape ρ_j. The adversary $\mathcal{A}$ is invoked in the real execution with the same random tape ρ_J and the correct inputs $(x_j)_{j\notin I}$ of the honest parties in the real execution.
The hybrid execution with $\mathcal{S}_2$ is identical to the hybrid execution with $\mathcal{S}_1$ with one distinction: $\mathcal{S}_2$ simulates the round ORR messages of the honest parties using their next-message functions. However, observe that $\mathcal{S}_2$ simulates the honest parties with the correct inputs. Hence, for a fixed randomness of the honest parties, the view of the adversary in π is exactly the same as the view of the simulator $\mathcal{S}_2$.
We now turn to the outputs of the honest parties in both executions. Note that, the messages sent by $\mathcal{A}$ in both the hybrid and real executions must be exactly the same. Hence, the transcript of messages (between all parties) in the execution of $\mathcal{S}_2$ are exactly the same as the transcript of messages (between all parties) in the execution of π. Hence, the outputs of the simulated honest parties are identical to the outputs of the honest parties in the protocol π. From the perfect security of π, we have that the outputs of the honest parties in π are identical to $f_J(\boldsymbol{x})$. Hence, we have that the outputs of the honest parties in the hybrid execution with $\mathcal{S}_2$ are identical to the outputs of the honest parties in the real execution. As a result, the hybrid execution with $\mathcal{S}_2$ is identical to the real execution of the protocol π.

From the above we can thus conclude that for every ρ_J

$$\text{HYB}_1\left(\mathcal{S}_1^{\mathcal{A}(\rho_J,(x_j)_{j\notin I})}(\rho_J)\right) \equiv \text{HYB}_2\left(\mathcal{S}_1^{\mathcal{A}(\rho_J,(x_j)_{j\notin I})}(\rho_J)\right) \ ,$$

and as a result, it must hold that

$$\text{HYB}_1\left(\mathcal{S}_1'^{\mathcal{A}'}(\rho_J)\right) \equiv \text{HYB}_2\left(\mathcal{S}_2'^{\mathcal{A}'}(\rho_J)\right) \ ,$$

HYB_1 and HYB_2 are identically distributed. □

Claim 6.11. HYB_2 *and* HYB_3 *are identically distributed.*

Proof. Let $\mathcal{S}'_2$ and $\mathcal{S}_2$ be the simulators defined in HYB_2 and let $\mathcal{S}'_3$ and $\mathcal{S}_3$ be the simulators defined in HYB_3. In HYB_2, the simulators $\mathcal{S}'_2$ and $\mathcal{S}_2$ receive the actual inputs $(x_j)_{j\notin I}$ of the honest parties and simulate the execution of the entire protocol with the adversary (super-rushing and rushing respectively). Even in HYB_3, the simulators $\mathcal{S}'_3$ and $\mathcal{S}_3$ perform the same as $\mathcal{S}'_2$ and $\mathcal{S}_2$ respectively.

However, the difference is that in HYB_2, we consider the outputs of the honest parties as computed by the trusted party, whereas, in HYB_3, we consider the outputs of the simulated honest parties. Let each of the simulators $\mathcal{S}_2', \mathcal{S}_2, \mathcal{S}_3', \mathcal{S}_3$ take as input: $(z, I, \boldsymbol{x}; (\rho_j)_{j \notin I})$, where we make explicit the random tapes $(\rho_j)_{j \notin I}$ of the simulated honest parties. Let $\mathcal{A}$ be the rushing adversary constructed from a fixed super-rushing adversary $\mathcal{A}'$ (see Adversary 6.8).

We complete the proof with the following steps:

1. For each $\rho_J = (\rho_j)_{j \notin I}$, we claim that

$$\text{HYB}_2\left(\mathcal{S}_2'^{\mathcal{A}'}(\rho_J)\right) \equiv \text{HYB}_2\left(\mathcal{S}_2^{\mathcal{A}(\rho_J,(x_j)_{j\notin I})}(\rho_J)\right) ,$$

   where $\mathcal{S}_2'$ with random tape ρ_J for the simulated honest parties, invokes $\mathcal{S}_2$ and $\mathcal{A}$ with the same random tapes ρ_J for the honest parties. In addition, $\mathcal{A}$ is invoked with the random inputs $(x_j')_{j \notin I}$ set as the actual inputs $(x_j)_{j \notin I}$ of the honest parties. We already show this in the proof of Claim 6.10.

2. For each ρ_J, we also claim that

$$\text{HYB}_3\left(\mathcal{S}_3'^{\mathcal{A}'}\right) \equiv \text{HYB}_3\left(\mathcal{S}_3^{\mathcal{A}(\rho_J,(x_j)_{j\notin I})}(\rho_J)\right) ,$$

   where $\mathcal{S}_3'$ with random tape ρ_J for the simulated honest parties, invokes $\mathcal{S}_3$ and $\mathcal{A}$ with the same random tapes ρ_J for the honest parties. In addition, $\mathcal{A}$ is invoked with the random inputs $(x_j')_{j \notin I}$ set as the actual inputs $(x_j)_{j \notin I}$ of the honest parties.
   Observe that $\mathcal{S}_3'$ executes exactly the same steps as $\mathcal{S}_3$ invoked with $\mathcal{A}$. The random tapes and the inputs for the simulated honest parties are identical in the execution of $\mathcal{S}_1'$ and the execution of $\mathcal{S}_1$ with $\mathcal{A}$. Additionally, the messages sent by $\mathcal{A}$ are identical to the messages sent by $\mathcal{A}'$. These changes are syntactic and hence the view of $\mathcal{A}'$ in the first execution is identical to the view of $\mathcal{A}$ in the second execution. Furthermore, $\mathcal{S}_3'$ uses $\mathcal{S}_3$ to extract the inputs $(x_i')_{i \in I}$ of the corrupted parties after round $\mathsf{CR}+1$. The trusted party computes $(y_1, \ldots, y_n) = f(x_1', \ldots, x_n')$ where $x_j' = x_j$ is the actual input of each honest party P_j. Both $\mathcal{S}_1'$ and $\mathcal{S}_1$ receive $(y_i)_{i \in I}$ and use SimOut to simulate the messages in round ORR (note that from Claim 6.5, we have that $\mathcal{S}_3'$ has already performed the input extraction and stored the outputs $(y_i)_{i \in I}$). Also, the trusted party computes f on the same inputs in both hybrid executions we consider.
   All the above are syntactic changes and as a result, on randomness ρ_J, the hybrid execution with $\mathcal{S}_3'$ is identical to the hybrid execution with $\mathcal{S}_3$ with $\mathcal{A}$.

3. Now, we claim for each ρ_J, that

$$\text{HYB}_2\left(\mathcal{S}_2^{\mathcal{A}(\rho_J,(x_j)_{j\notin I})}(\rho_J)\right) \equiv \left\{\text{REAL}_{\pi(\rho_J,(x_j)_{j\notin I}),\mathcal{A}(\rho_J,(x_j)_{j\notin I}),I}(\boldsymbol{x})\right\} ,$$

   In the hybrid execution, $\mathcal{S}_2$ is invoked with $\mathcal{A}$ with the same random tape ρ_j for each honest party P_j. Furthermore, $\mathcal{A}$ is invoked with the random inputs $(x_j')_{j \notin I}$ set as the actual inputs $(x_j)_{j \notin I}$ of the honest parties that are received by $\mathcal{S}_2$ as input. In the real execution, π is executed with each honest party P_j for $j \notin I$ holding input x_j and random tape ρ_j. The adversary $\mathcal{A}$ is invoked in the real execution with the same random tape ρ_J and the correct inputs $(x_j)_{j \notin I}$ of the honest parties in the real execution. We already show this in the proof of Claim 6.10.

4. Now, we claim for each ρ_J, that

$$\text{HYB}_3\left(\mathcal{S}_3^{\mathcal{A}(\rho_J,(x_j)_{j\notin I})}(\rho_J)\right) \equiv \left\{\text{REAL}_{\pi(\rho_J,(x_j)_{j\notin I}),\mathcal{A}(\rho_J,(x_j)_{j\notin I}),I}(\boldsymbol{x})\right\} .$$

In the hybrid execution, $\mathcal{S}_3$ is invoked with $\mathcal{A}$ with the same random tape ρ_j for each honest party P_j. Furthermore, $\mathcal{A}$ is invoked with the random inputs $(x'_j)_{j\notin I}$ set as the actual inputs $(x_j)_{j\notin I}$ of the honest parties that are received by $\mathcal{S}_3$ as input. In the real execution, π is executed with each honest party P_j for $j \notin I$ holding input x_j and random tape ρ_j. The adversary $\mathcal{A}$ is invoked in the real execution with the same random tape ρ_J and the correct inputs $(x_j)_{j\notin I}$ of the honest parties in the real execution.
The hybrid execution with $\mathcal{S}_3$ is identical to the execution of the protocol π, as the randomness is fixed and the simulated honest parties start with the correct inputs. Hence, for a fixed randomness of the honest parties, the view of the adversary $\mathcal{A}$ in π is exactly the same as the view of the simulator $\mathcal{S}_3$.
We now turn to the outputs of the honest parties in both executions. Note that, the messages sent by $\mathcal{A}$ in both the hybrid and real executions must be exactly the same. Hence, the transcript of messages (between all parties) in the execution of $\mathcal{S}_3$ are exactly the same as the transcript of messages (between all parties) in the execution of π. Hence, the outputs of the simulated honest parties are identical to the outputs of the honest parties in the protocol π. From the perfect security of π, we have that the outputs of the honest parties in π are identical to $f_J(\boldsymbol{x})$. Hence, we have that the outputs of the simulated honest parties in the hybrid execution with $\mathcal{S}_3$ are identical to the outputs of the honest parties in the real execution.

From the above we can thus conclude that for every ρ_J

$$\text{HYB}_2\left(\mathcal{S}_2^{\mathcal{A}(\rho_J,(x_j)_{j\notin I})}(\rho_J)\right) \equiv \text{HYB}_3\left(\mathcal{S}_3^{\mathcal{A}(\rho_J,(x_j)_{j\notin I})}(\rho_J)\right) \ ,$$

and as a result, it must hold that

$$\text{HYB}_2\left(\mathcal{S}_2'^{\mathcal{A}'}(\rho_J)\right) \equiv \text{HYB}_3\left(\mathcal{S}_3'^{\mathcal{A}'}(\rho_J)\right) \ ,$$

Hence, HYB_2 and HYB_3 are identically distributed. □

HYB_3 and HYB_4 are just a syntactic change, and the distributions must be identical. Finally, we have that for every super-rushing adversary $\mathcal{A}'$, there exists a simulator $\mathcal{S}'$ such that

$$\left\{\text{REAL}_{\pi,\mathcal{A}',I}(\boldsymbol{x})\right\} \equiv \left\{\text{IDEAL}_{f,\mathcal{S}',I}(\boldsymbol{x})\right\} \ .$$

□

6.5 Extensions

In this section, we show extensions of Theorem 6.3. Before proceeding, we note that our definition of a committal round CR and an output revealing round ORR, implicitly requires that $\mathsf{ORR} > \mathsf{CR}$.

Input committal synchronization. Suppose that we have a protocol π that securely realizes a functionality f with compatible simulation but for which $\mathsf{ORR} = \mathsf{CR} + 1$. Consider the following protocol π' that is identical to π with the following instruction added for each party after the committal round CR:

1. If all round CR messages of the protocol π are received, then send ready to all the parties.
2. Proceed to round $\mathsf{CR}+1$ of π only after receiving ready from all parties.

This results in a protocol where $\mathsf{ORR} > \mathsf{CR} + 1$. We term this transformation, input committal synchronization and state the following corollary.

Corollary 6.12. *Let f be an n-ary functionality, and suppose that the protocol π satisfies all the requirements of Theorem 6.3, except for $\mathsf{ORR} > \mathsf{CR} + 1$. Then, the protocol π' with* input committal synchronization *securely realizes f with perfect security against super-rushing adversaries.*

Committal round over broadcast (a special case of [BGW88]). Consider the BGW protocol for the computation of a linear function. This protocol satisfies all of our sufficient conditions except the condition that $\mathsf{ORR} > \mathsf{CR} + 1$. To see this, observe that the BGW protocol for the computation of linear functions is as follows:

1. Each party with an input, executes a VSS of its input.
2. The parties perform a local computation on the shares they have received from all the VSS instances to obtain shares on the output wires.
3. The parties execute a reconstruction of the outputs by reconstructing the sharings on the output wires.

The protocol admits a committal round (the last round of the VSS) and an output revealing round that immediately follows the committal round. Hence, for this protocol, $\mathsf{ORR} = \mathsf{CR} + 1$. Note that the same holds true for the perfectly-secure protocols of [ABT19, AKP20].

We can derive the security of BGW for linear functions against super-rushing adversaries using Corollary 6.12 at the cost of incurring an additional round. However, we observe that the committal round CR of this protocol is over the broadcast channel. Therefore, if an honest party completes CR, i.e., if the adversary provided all messages to some honest P_{j^*} and requires to see its round $\mathsf{CR} + 1$ messages, then the view of all other honest parties P_j in round $\mathsf{CR} + 1$ is also determined and must be the same set of messages.

Corollary 6.13. *Let f be an n-ary functionality, and suppose that π satisfies all the requirements of Theorem 6.3, except for $\mathsf{ORR} > \mathsf{CR} + 1$, however, CR occurs over the broadcast channel. Then, the protocol π securely realizes f with perfect security against super-rushing adversaries.*

Proof. Consider the simulator $\mathcal{S}'$ we construct in proof of Theorem 6.3. We replace Step 5c with the following step:

5. **Interaction with $\mathcal{A}'$ for every $r = 1, \ldots, \mathsf{ORR}$:** Send and receive messages from $\mathcal{A}'$ as follows:
   (c) If $r = 1, \ldots, \mathsf{ORR} - 1$ and there is no message $(j^*, i^*, r, m^r_{j^* \to i^*})$ in M_r: Invoke the next-message function of the virtual honest party P_{j^*} on all its incoming messages at round $r - 1$. Add all the messages to M_r.
   If $r = \mathsf{CR} + 1$, then
      i. For each $i \in I$ and $j \notin I$, such that the message $(i, j, \mathsf{CR}, m^{\mathsf{CR}}_{i \to j}) \notin M_{\mathsf{CR}}$, add the message $(i, j, \mathsf{CR}, m^{\mathsf{CR}}_{i \to j})$ to M_{CR} where $m^{\mathsf{CR}}_{i \to j} = m^{\mathsf{CR}}_{i \to j^*}$.
      ii. Invoke the next-message function of each virtual honest part P_{j^*} on all its incoming messages at round CR and add these messages to $M_{\mathsf{CR}+1}$.

Suppose that the super-rushing adversary $\mathcal{A}'$ requests to receive a message $(j^*, i^*, \mathsf{CR} + 1)$ before it has provided all the round CR messages to the other honest parties. $\mathcal{A}'$ must have provided P_{j^*} the messages $(i, j^*, \mathsf{CR}, m^{\mathsf{CR}}_{i \to j^*})$ for every $i \in I$, as otherwise it is not a valid adversary. Moreover, recall that we assume that $\mathsf{ORR} = \mathsf{CR} + 1$ and the messages from the simulated honest party P_{j^*} must be computed using SimOut_{j^*}.

Since round CR is over broadcast it must hold for each $i \in I$ and $j, k \notin I$ that $m^{\mathsf{CR}}_{i \to j} = m^{\mathsf{CR}}_{i \to k}$. Hence, our modified $\mathcal{S}'$ (constructed above) adds these messages to all honest parties to CR. Therefore, when the first simulated honest party P_{j^*} completes round CR, the simulator $\mathcal{S}'$ immediately completes round CR for all simulated honest parties.

As a consequence, $\mathcal{S}'$ invokes Step 6b when the first simulated honest party completes round CR and stores the outputs of the corrupted parties. Therefore, $\mathcal{S}'$ does not abort due to the $\mathsf{noOutputs}$ event in Step 5(d)i in the execution of round $\mathsf{ORR} = \mathsf{CR} + 1$. The rest of the proof carries over as is. □

7 A Separation for the Statistical Setting

Let f be an n-ary functionality and let π be a protocol that satisfies all our sufficient conditions in Theorem 6.3, except that π achieves only statistical security against rushing adversaries (see Definition 3.4). We show that security against super-rushing adversaries is not always implied by security against rushing adversaries.

To that end, consider the BGW protocol for $n = 5$ parties with $t = 1$ corruptions (which satisfies all our sufficient conditions, including perfect security) with the following modifications:

1. The parties first execute an instance of the $(2, 5)$ simultaneous broadcast protocol $\pi_{\mathsf{sb\text{-}basic}}$ (from Section 4.1.1) on uniformly random inputs $r_1, r_2 \leftarrow \{0,1\}^\kappa$ for parties P_1 and P_2, respectively, where κ denotes the statistical security parameter.[7] Let (y_1, y_2) denote the output of the parties in this phase.
2. If $y_1 = y_2$, the parties reveal their private inputs.
3. The parties then execute the BGW protocol on their actual inputs.

The above protocol securely computes f with perfect security against rushing adversaries conditioned on $y_1 \neq y_2$. From Lemma 4.2, we have that $\pi_{\mathsf{sb\text{-}basic}}$ is perfectly secure in the presence of a rushing adversary. Therefore, the input r_1 of P_1 is independent of the input r_2 of P_2, and vice versa, even if one of the parties is cheating. As a result, $\mathsf{Pr}[y_1 = y_2] \leq 2^{-\kappa}$ in the presence of a rushing adversary and the modified BGW protocol securely realizes f with statistical security in the presence of a rushing adversary. In fact, the simulator can completely ignore the bad event in the simulation, and the statistical distance would be $2^{-\kappa}$. As such, the modified protocol still admits a committal round and an output revealing round.

By Claim 4.4, there exists a super-rushing adversary to the protocol $\pi_{\mathsf{sb\text{-}basic}}$ that can enforce $\Pr[y_1 = y_2] = 1$. As a result, in the real execution of the modified BGW protocol, there exists a super-rushing adversary that learns the inputs of the honest parties. Clearly, such behavior cannot be simulated by any simulator in the ideal execution; therefore, the above protocol is not secure against a super-rushing adversary.

Acknowledgments The authors thank Eyal Kushilevitz for interesting discussions on super-rushing adversaries.

[7] Although the protocol $\pi_{\mathsf{sb\text{-}basic}}$ is defined in Section 4.1.1) for bits, it can operate in a straightforward way over longer strings.

References

[AAPP23] Ittai Abraham, Gilad Asharov, Shravani Patil, and Arpita Patra. Detect, pack and batch: Perfectly-secure MPC with linear communication and constant expected time. In *EUROCRYPT '23, Part II*, volume 14005, pages 251–281, 2023.

[AAPP24] Ittai Abraham, Gilad Asharov, Shravani Patil, and Arpita Patra. Perfect asynchronous MPC with linear communication overhead. In *EUROCRYPT '24, Part V*, volume 14655, pages 280–309, 2024.

[AAY21] Ittai Abraham, Gilad Asharov, and Avishay Yanai. Efficient perfectly secure computation with optimal resilience. In *TCC '21*, pages 66–96, 2021.

[ABT19] Benny Applebaum, Zvika Brakerski, and Rotem Tsabary. Degree 2 is complete for the round-complexity of malicious MPC. In *EUROCRYPT '19, Part II*, volume 11477, pages 504–531, 2019.

[ACC25] Ananya Appan, Anirudh Chandramouli, and Ashish Choudhury. Network agnostic perfectly secure multiparty computation against general adversaries. *IEEE Trans. Inf. Theory*, 71(1):644–682, 2025.

[ACS22] Gilad Asharov, Ran Cohen, and Oren Shochat. Static vs. adaptive security in perfect MPC: A separation and the adaptive security of BGW. In *ITC '22*, volume 230, pages 15:1–15:16, 2022.

[ADLS91] Hagit Attiya, Cynthia Dwork, Nancy A. Lynch, and Larry J. Stockmeyer. Bounds on the time to reach agreement in the presence of timing uncertainty. In *STOC '91*, pages 359–369, 1991.

[AKP20] Benny Applebaum, Eliran Kachlon, and Arpita Patra. The round complexity of perfect mpc with active security and optimal resiliency. In *FOCS '20*, pages 1277–1284, 2020.

[AL17] Gilad Asharov and Yehuda Lindell. A full proof of the bgw protocol for perfectly secure multiparty computation. *Journal of Cryptology*, 30(1):58–151, 2017.

[ALR11] Gilad Asharov, Yehuda Lindell, and Tal Rabin. Perfectly-secure multiplication for any $t < n/3$. In *CRYPTO '11*, pages 240–258, 2011.

[BB89] Judit Bar-Ilan and Donald Beaver. Non-cryptographic fault-tolerant computing in constant number of rounds of interaction. In Piotr Rudnicki, editor, *Proceedings of the Eighth Annual ACM Symposium on Principles of Distributed Computing, Edmonton, Alberta, Canada, August 14-16, 1989*, pages 201–209. ACM, 1989.

[BCG93] Michael Ben-Or, Ran Canetti, and Oded Goldreich. Asynchronous secure computation. In *STOC '93*, pages 52–61, 1993.

[BCLL23] Renas Bacho, Daniel Collins, Chen-Da Liu-Zhang, and Julian Loss. Network-agnostic security comes (almost) for free in DKG and MPC. In *CRYPTO '23, Part I*, volume 14081, pages 71–106, 2023.

[BDD+21] Carsten Baum, Bernardo David, Rafael Dowsley, Jesper Buus Nielsen, and Sabine Oechsner. TARDIS: A foundation of time-lock puzzles in UC. In *EUROCRYPT '21, Part III*, volume 12698, pages 429–459, 2021.

[Bea91] Donald Beaver. Foundations of secure interactive computing. In *CRYPTO '91*, volume 576, pages 377–391, 1991.

[BGW88] Michael Ben-Or, Shafi Goldwasser, and Avi Wigderson. Completeness theorems for non-cryptographic fault-tolerant distributed computation (extended abstract). In *STOC '88*, pages 1–10, 1988.

[BH08] Zuzana Beerliová-Trubíniová and Martin Hirt. Perfectly-secure mpc with linear communication complexity. In *TCC '08*, 2008.

[BHN10] Zuzana Beerliová-Trubíniová, Martin Hirt, and Jesper Buus Nielsen. On the theoretical gap between synchronous and asynchronous MPC protocols. In *PODC '10*, pages 211–218, 2010.

[BKL19] Erica Blum, Jonathan Katz, and Julian Loss. Synchronous consensus with optimal asynchronous fallback guarantees. In *TCC '19, Part I*, volume 11891, pages 131–150, 2019.

[BKR94] Michael Ben-Or, Boaz Kelmer, and Tal Rabin. Asynchronous secure computations with optimal resilience (extended abstract). In *PODC '94*, pages 183–192, 1994.

[BZL20] Erica Blum, Chen-Da Liu Zhang, and Julian Loss. Always have a backup plan: Fully secure synchronous MPC with asynchronous fallback. In *CRYPTO '20, Part II*, volume 12171, pages 707–731, 2020.

[Can00] Ran Canetti. Security and composition of multiparty cryptographic protocols. *J. Cryptol.*, 13(1):143–202, 2000.

[Can01] Ran Canetti. Universally composable security: A new paradigm for cryptographic protocols. In *FOCS '01*, pages 136–145, 2001.

[CCD88] David Chaum, Claude Crépeau, and Ivan Damgård. Multiparty unconditionally secure protocols (extended abstract). In *STOC '88*, pages 11–19, 1988.

[CCGZ19] Ran Cohen, Sandro Coretti, Juan A. Garay, and Vassilis Zikas. Probabilistic termination and composability of cryptographic protocols. *J. Cryptol.*, 32(3):690–741, 2019.

[CCGZ21] Ran Cohen, Sandro Coretti, Juan A. Garay, and Vassilis Zikas. Round-preserving parallel composition of probabilistic-termination cryptographic protocols. *J. Cryptol.*, 34(2):12, 2021.

[CCXY18] Ignacio Cascudo, Ronald Cramer, Chaoping Xing, and Chen Yuan. Amortized complexity of information-theoretically secure MPC revisited. In Hovav Shacham and Alexandra Boldyreva, editors, *Advances in Cryptology - CRYPTO 2018 - 38th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 19-23, 2018, Proceedings, Part III*, volume 10993 of *Lecture Notes in Computer Science*, pages 395–426. Springer, 2018.

[CDD+01] Ran Canetti, Ivan Damgård, Stefan Dziembowski, Yuval Ishai, and Tal Malkin. On adaptive vs. non-adaptive security of multiparty protocols. In *EUROCRYPT '01*, pages 262–279, 2001.

[CDM00] Ronald Cramer, Ivan Damgård, and Ueli M. Maurer. General secure multi-party computation from any linear secret-sharing scheme. In *EUROCRYPT '00*, volume 1807, pages 316–334, 2000.

[CGHZ16] Sandro Coretti, Juan A. Garay, Martin Hirt, and Vassilis Zikas. Constant-round asynchronous multi-party computation based on one-way functions. In *ASIACRYPT '16, Part II*, volume 10032, pages 998–1021, 2016.

[CGMA85] Benny Chor, Shafi Goldwasser, Silvio Micali, and Baruch Awerbuch. Verifiable secret sharing and achieving simultaneity in the presence of faults (extended abstract). In *FOCS '85*, pages 383–395, 1985.

[CK93] Benny Chor and Eyal Kushilevitz. A communication-privacy tradeoff for modular addition. *Inf. Process. Lett.*, 45(4):205–210, 1993.

[CMS85] Benny Chor, Michael Merritt, and David B. Shmoys. Simple constant-time consensus protocols in realistic failure models (extended abstract). In *PODC '85*, pages 152–162, 1985.

[Coh16] Ran Cohen. Asynchronous secure multiparty computation in constant time. In *PKC '16, Part II*, volume 9615, pages 183–207, 2016.

[CP17] Ashish Choudhury and Arpita Patra. An efficient framework for unconditionally secure multiparty computation. *IEEE Trans. Inf. Theory*, 63(1):428–468, 2017.

[DLS88] Cynthia Dwork, Nancy A. Lynch, and Larry J. Stockmeyer. Consensus in the presence of partial synchrony. *J. ACM*, 35(2):288–323, 1988.

[DM00] Yevgeniy Dodis and Silvio Micali. Parallel reducibility for information-theoretically secure computation. In *CRYPTO '00*, volume 1880, pages 74–92, 2000.

[DN07] Ivan Damgård and Jesper Buus Nielsen. Scalable and unconditionally secure multiparty computation. In *CRYPTO '07*, volume 4622, pages 572–590, 2007.

[DN14] Ivan Damgård and Jesper Buus Nielsen. Adaptive versus static security in the UC model. In *ProvSec 2014*, volume 8782 of *Lecture Notes in Computer Science*, pages 10–28. Springer, 2014.

[FLP85] Michael J. Fischer, Nancy A. Lynch, and Mike Paterson. Impossibility of distributed consensus with one faulty process. *J. ACM*, 32(2):374–382, 1985.

[FY92] Matthew K. Franklin and Moti Yung. Communication complexity of secure computation (extended abstract). In S. Rao Kosaraju, Mike Fellows, Avi Wigderson, and John A. Ellis, editors, *Proceedings of the 24th Annual ACM Symposium on Theory of Computing, May 4-6, 1992, Victoria, British Columbia, Canada*, pages 699–710. ACM, 1992.

[GIKR01] Rosario Gennaro, Yuval Ishai, Eyal Kushilevitz, and Tal Rabin. The round complexity of verifiable secret sharing and secure multicast. In *STOC '01*, pages 580–589, 2001.

[GIKR02] Rosario Gennaro, Yuval Ishai, Eyal Kushilevitz, and Tal Rabin. On 2-round secure multiparty computation. In *CRYPTO '02*, volume 2442, pages 178–193, 2002.

[GLS19] Vipul Goyal, Yanyi Liu, and Yifan Song. Communication-efficient unconditional mpc with guaranteed output delivery. In *CRYPTO '19, Part II*, pages 85–114, 2019.

[GMW87] Oded Goldreich, Silvio Micali, and Avi Wigderson. How to play any mental game or A completeness theorem for protocols with honest majority. In *STOC '87*, pages 218–229, 1987.

[Gol04] Oded Goldreich. *The Foundations of Cryptography - Volume 2: Basic Applications*. Cambridge University Press, 2004.

[GRR98] Rosario Gennaro, Michael O. Rabin, and Tal Rabin. Simplified VSS and fast-track multiparty computations with applications to threshold cryptography. In Brian A. Coan and Yehuda Afek, editors, *Proceedings of the Seventeenth Annual ACM Symposium on Principles of Distributed Computing, PODC '98, Puerto Vallarta, Mexico, June 28 - July 2, 1998*, pages 101–111. ACM, 1998.

[HM97] Martin Hirt and Ueli M. Maurer. Complete characterization of adversaries tolerable in secure multi-party computation (extended abstract). In James E. Burns and Hagit Attiya, editors, *Proceedings of the Sixteenth Annual ACM Symposium on Principles of Distributed Computing, Santa Barbara, California, USA, August 21-24, 1997*, pages 25–34. ACM, 1997.

[HM00] Martin Hirt and Ueli M. Maurer. Player simulation and general adversary structures in perfect multiparty computation. *J. Cryptol.*, 13(1):31–60, 2000.

[HM01] Martin Hirt and Ueli M. Maurer. Robustness for free in unconditional multi-party computation. In Joe Kilian, editor, *Advances in Cryptology - CRYPTO 2001, 21st Annual International Cryptology Conference, Santa Barbara, California, USA, August 19-23, 2001, Proceedings*, volume 2139 of *Lecture Notes in Computer Science*, pages 101–118. Springer, 2001.

[HM04] Dennis Hofheinz and Jörn Müller-Quade. A synchronous model for multi-party computation and the incompleteness of oblivious transfer. *IACR Cryptol. ePrint Arch.*, page 16, 2004.

[HMP00] Martin Hirt, Ueli M. Maurer, and Bartosz Przydatek. Efficient secure multi-party computation. In Tatsuaki Okamoto, editor, *Advances in Cryptology - ASIACRYPT 2000, 6th International Conference on the Theory and Application of Cryptology and Information Security, Kyoto, Japan, December 3-7, 2000, Proceedings*, volume 1976 of *Lecture Notes in Computer Science*, pages 143–161. Springer, 2000.

[HNP05] Martin Hirt, Jesper Buus Nielsen, and Bartosz Przydatek. Cryptographic asynchronous multiparty computation with optimal resilience (extended abstract). In *EUROCRYPT '05*, volume 3494, pages 322–340, 2005.

[KLP07] Yael Tauman Kalai, Yehuda Lindell, and Manoj Prabhakaran. Concurrent composition of secure protocols in the timing model. *J. Cryptol.*, 20(4):431–492, 2007.

[KLR06] Eyal Kushilevitz, Yehuda Lindell, and Tal Rabin. Information-theoretically secure protocols and security under composition. In *STOC '06*, pages 109–118, 2006.

[KMTZ13] Jonathan Katz, Ueli Maurer, Björn Tackmann, and Vassilis Zikas. Universally composable synchronous computation. In *TCC '13*, volume 7785, pages 477–498, 2013.

[LLM+20] Chen-Da Liu-Zhang, Julian Loss, Ueli Maurer, Tal Moran, and Daniel Tschudi. MPC with synchronous security and asynchronous responsiveness. In *ASIACRYPT '20, Part III*, volume 12493, pages 92–119, 2020.

[LM20] Chen-Da Liu-Zhang and Ueli Maurer. Synchronous constructive cryptography. In *TCC '20,Part II*, volume 12551, pages 439–472, 2020.

[LSP82] Leslie Lamport, Robert E. Shostak, and Marshall C. Pease. The Byzantine generals problem. *ACM Trans. Program. Lang. Syst.*, 4(3):382–401, 1982.

[MR91] Silvio Micali and Phillip Rogaway. Secure computation (abstract). In *CRYPTO '91*, volume 576, pages 392–404, 1991.

[MR92] Silvio Micali and Phillip Rogaway. Secure computation (unpublished manuscript), 1992. Preliminary version in [MR91]. (All references within refer to the unpublished manuscript.).

[Nie03] Jesper Buus Nielsen. *On Protocol Security in the Cryptographic Model*. PhD thesis, University of Aarhus, 2003.

[PS17] Rafael Pass and Elaine Shi. Hybrid consensus: Efficient consensus in the permissionless model. In *DISC '17*, volume 91, pages 39:1–39:16, 2017.

[PS18] Rafael Pass and Elaine Shi. Thunderella: Blockchains with optimistic instant confirmation. In *EUROCRYPT '18*, volume 10821, pages 3–33, 2018.

[PSL80] Marshall C. Pease, Robert E. Shostak, and Leslie Lamport. Reaching agreement in the presence of faults. *J. ACM*, 27(2):228–234, 1980.

[SARN20] Nibesh Shrestha, Ittai Abraham, Ling Ren, and Kartik Nayak. On the optimality of optimistic responsiveness. In *CCS '20*, pages 839–857, 2020.

[Yao86] Andrew Chi-Chih Yao. How to generate and exchange secrets (extended abstract). In *FOCS '86*, pages 160–164, 1986.